GIT CLI

CHEAT SHEETS

A Complete Reference for the Git Command Line

Setup & Config · Staging & Commits · History & Diffs · Branches & Tags · Merge & Rebase · Remotes & PRs · Undo & Recovery · Stash · Bisect & Worktrees · Submodules & LFS · Hooks · Quick Reference

36 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & Mental Model
02 Installation & First-Time Config
03 Config Levels, Aliases & Help

STARTING A REPOSITORY

04 Init & Clone
05 Ignoring Files & .gitattributes

EVERYDAY WORKFLOW

06 Status & Staging
07 Committing
08 Viewing Diffs
09 Discarding Working Changes

HISTORY & INSPECTION

10 Log Basics
11 Log Filtering & Formatting
12 Referencing Commits
13 Show, Blame & Grep

BRANCHES & TAGS

14 Creating & Managing Branches
15 Tags

MERGING & REBASING

16 Merging
17 Resolving Conflicts
18 Rebasing
19 Interactive Rebase
20 Cherry-Pick & Patches

REMOTES & COLLABORATION

21 Remotes, Fetch & Pull
22 Push
23 SSH, Credentials & Signing
24 Collaboration Workflows

UNDOING & RECOVERY

25 Reset
26 Revert vs Reset vs Restore
27 Stash
28 Reflog & Recovery
29 Common Errors & Fixes

ADVANCED TOOLS

30 Bisect & Worktrees
31 Submodules & Subtrees
32 Large Repos & Big Files
33 Hooks
34 Rewriting History & Removing Secrets

QUICK REFERENCE

35 Quick Reference — Everyday Commands
36 Quick Reference — Merge, Undo & Advanced

GIT

Table of Contents

36 sheets · syntax · examples · notes

INTRODUCTION & SETUP

01 Introduction & Mental Model
02 Installation & First-Time Config
03 Config Levels, Aliases & Help

STARTING A REPOSITORY

04 Init & Clone
05 Ignoring Files & .gitattributes

EVERYDAY WORKFLOW

06 Status & Staging
07 Committing
08 Viewing Diffs
09 Discarding Working Changes

HISTORY & INSPECTION

10 Log Basics
11 Log Filtering & Formatting
12 Referencing Commits
13 Show, Blame & Grep

BRANCHES & TAGS

14 Creating & Managing Branches
15 Tags

MERGING & REBASING

16 Merging
17 Resolving Conflicts
18 Rebasing
19 Interactive Rebase
20 Cherry-Pick & Patches

REMOTES & COLLABORATION

21 Remotes, Fetch & Pull
22 Push
23 SSH, Credentials & Signing
24 Collaboration Workflows

UNDOING & RECOVERY

25 Reset
26 Revert vs Reset vs Restore
27 Stash
28 Reflog & Recovery
29 Common Errors & Fixes

ADVANCED TOOLS

30 Bisect & Worktrees
31 Submodules & Subtrees
32 Large Repos & Big Files
33 Hooks
34 Rewriting History & Removing Secrets

QUICK REFERENCE

35 Quick Reference — Everyday Commands
36 Quick Reference — Merge, Undo & Advanced

Introduction & Mental Model

WHAT GIT IS

Git is a **distributed version control system** created by Linus Torvalds in 2005 for Linux kernel development. Every clone holds the **full history**, so almost every command runs locally and offline. Git stores **snapshots**, not diffs: each commit records the whole project tree, and unchanged files are shared between commits. Learn the three areas below and most commands become obvious.

KEY FACTS

- Every object is named by a hash of its content: SHA-1 today, SHA-256 optional
- `HEAD` points to the branch you are on, and that branch points to a commit
- Branches are 40-byte files holding a hash, so they are nearly free to create
- Git 3.0 (planned): `main` default branch, SHA-256 for new repos, reftable refs

HELLO, GIT — A FIRST REPOSITORY

```
mkdir demo && cd demo
git init                        # creates .git/ (the repository)
echo "# Demo" > README.md       # file exists only in the working tree
git status                      # README.md is "untracked"
git add README.md               # copy the snapshot into the staging area
git commit -m "Add README"      # store the staged snapshot as a commit
git log --oneline               # a1b2c3d (HEAD -> main) Add README

# Look inside the commit you just made
git cat-file -t HEAD            # commit
git cat-file -p HEAD            # tree <sha>, author, committer, message
git cat-file -p HEAD^{tree}     # 100644 blob <sha>  README.md
```

CODE EXPLAINED

- → `git init` — creates the `.git/` folder that holds every object and ref
- → `git add` — stages a snapshot of the file's current content
- → `git commit` — saves the staged snapshot with author, date, and message
- → `git cat-file -t` — prints an object's type
- → `git cat-file -p` — pretty-prints an object's content
- → `HEAD^{tree}` — the tree object of the current commit

THE THREE AREAS

AREA	WHAT LIVES THERE
`Working tree`	The files you edit on disk
`Staging area`	The index: the snapshot for your next commit
`Repository`	`.git/`: commits, objects, refs
`Remote`	Another copy of the repo, e.g. `origin`

OBJECT TYPES

OBJECT	STORES
`blob`	File content (no name, no mode)
`tree`	A directory: names, modes, blob/tree hashes
`commit`	Tree hash, parents, author, message
`tag`	Annotated tag: target, tagger, message

FILE STATES

STATE	MEANING
`untracked`	Not in the last commit or index
`modified`	Changed since the last commit
`staged`	Change recorded in the index
`committed`	Safely stored in `.git/`

⚠ COMMON MISTAKES

- Thinking commits store diffs. They store full snapshots; diffs are computed on demand
- Treating Git and GitHub as the same thing. Git is the tool; GitHub hosts repos
- Deleting `.git/` to start over. That throws away all history in that clone
- Skipping `git status`. It tells you which area every change is in

✓ TIPS & BEST PRACTICES

- Run `git status` before and after every command while learning
- Use `git help <command>` for the full manual page offline
- Short hashes (7+ chars) work anywhere a full hash does
- Visualize history with `git log --oneline --graph --all`

QUICK SUMMARY

- `working tree` — files on disk you edit
- `index` — staging area for the next commit
- `commit` — snapshot + parents + message
- `HEAD` — the current branch / commit

Installation & First-Time Config

git --version · brew / apt / winget · user.name / user.email · init.defaultBranch main · core.editor · core.autocrlf

GETTING SET UP

Install Git from your platform's package manager, then set your **name and email** once. Git writes them into every commit you make, and hosts like GitHub use the email to link commits to your account. A few more global settings (default branch, editor, line endings) save headaches later. All of these live in `~/.gitconfig`.

NOTES

- macOS ships Apple Git via Xcode tools; Homebrew gives a newer release
- Ubuntu's default package can lag; the `git-core` PPA tracks the latest
- Git for Windows includes Git Bash and Git Credential Manager
- Use your host's no-reply email if you don't want your address public

INSTALL, VERIFY, CONFIGURE

```
brew install git                          # macOS (Homebrew)
sudo apt install git                      # Debian / Ubuntu
sudo dnf install git                      # Fedora / RHEL
winget install --id Git.Git -e            # Windows
git --version                             # git version 2.5x.x

git config --global user.name  "Ada Lovelace"
git config --global user.email "ada@example.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
git config --global core.autocrlf input   # macOS/Linux (Windows: true)
git config --global --list                # review everything you set
```

CODE EXPLAINED

- → `--global` — writes to `~/.gitconfig` for every repo on this account
- → `user.name` / `user.email` — stamped on each commit you create
- → `init.defaultBranch` — branch name for new `git init` repos
- → `core.editor` — opens for commit messages; `--wait` blocks until closed
- → `core.autocrlf` — converts line endings on commit/checkout
- → `--list` — prints all effective settings

CORE.EDITOR VALUES

VALUE	EDITOR
`"code --wait"`	VS Code
`nano`	Nano (beginner friendly)
`vim`	Vim (the usual default)
`"subl -n -w"`	Sublime Text
`"notepad"`	Windows Notepad

USEFUL GLOBAL SETTINGS

SETTING	EFFECT
`pull.rebase true`	`git pull` rebases instead of merging
`push.autoSetupRemote true`	first `push` sets upstream automatically
`fetch.prune true`	drop deleted remote branches on fetch
`rerere.enabled true`	remember conflict resolutions
`help.autocorrect prompt`	offer to run a mistyped command
`merge.conflictStyle zdiff3`	show the base in conflict blocks

⚠ COMMON MISTAKES

- Setting `user.email` per repo by accident (no `--global`). Other repos still use the old value
- Leaving the editor as Vim when you don't know Vim. `git commit` with no `-m` then feels stuck (`:wq` exits)
- `core.autocrlf true` on macOS/Linux. It rewrites line endings you didn't ask for

✓ TIPS & BEST PRACTICES

- Run `git config --global --edit` to edit the whole file at once
- Check one value with `git config user.email` inside a repo
- Keep work and personal identities apart with `includeIf` (Sheet 03)

QUICK SUMMARY

- `git --version` — confirm the install
- `user.name` / `user.email` — required identity
- `init.defaultBranch main` — modern default
- `core.editor` — editor for messages

Config Levels, Aliases & Help

WHERE SETTINGS LIVE

Git reads config from three levels and the **most specific one wins**: system, then global (your user), then local (this repo). **Aliases** are config entries too, so you can shorten long commands. When you forget an option, `git help` and –h work offline. A few CLI conventions, like –– and –C, apply to almost every command.

NOTES

- Local beats global beats system for the same key
- `--show-origin` tells you which file set a value
- Git 2.46+ adds subcommands: `git config get`, `set`, `list`
- `-c key=value` overrides config for a single command

CONFIG, ALIASES, HELP

```
git config --list --show-origin          # every value + the file it came from
git config --local user.email "me@work.com"  # this repo only
git config --global --unset pull.rebase   # remove a setting

# ~/.gitconfig: use a work identity for repos under ~/work/
[includeIf "gitdir:~/work/"]
    path = ~/.gitconfig-work

git config --global alias.st "status -sb"
git config --global alias.lg "log --oneline --graph --all --decorate"
git config --global alias.undo "reset --soft HEAD~1"
git config --global alias.amend '!git add -A && git commit --amend --no-edit'

git help rebase        # full manual page
git commit -h          # short option summary
git -C ~/code/app status   # run as if inside that folder
```

CODE EXPLAINED

- → `--show-origin` — prints `file:~/.gitconfig` next to each value
- → `includeIf "gitdir:…"` — loads another file for matching repos
- → `alias.st` — now `git st` runs `git status -sb`
- → `!` — alias runs a shell command, so you can chain several
- → `git help` — the man page; `-h` — a quick usage summary
- → `-C <path>` — run Git in another directory

CONFIG LEVELS

FLAG	FILE
`--system`	`/etc/gitconfig` (all users)
`--global`	`~/.gitconfig` or `~/.config/git/config`
`--local`	`.git/config` (default when writing)
`--worktree`	`.git/config.worktree`
`-c k=v`	one command only

GETTING HELP

COMMAND	SHOWS
`git help <cmd>`	full manual for a command
`git <cmd> -h`	short usage and options
`git help -a`	every available command
`git help -g`	concept guides (e.g. `revisions`)
`git help config`	every config key

CLI CONVENTIONS

SYNTAX	MEANING
`--`	everything after is a path, not a branch
`':!*.lock'`	pathspec: exclude matching files
`-C <dir>`	run in another directory
`--no-pager`	print without `less`
`-n / --dry-run`	preview (on add, clean, push)

⚠ COMMON MISTAKES

- Expecting `--global` to override a repo's `--local` value. Local always wins
- Forgetting the `!` on a shell alias. Without it Git treats the text as a git subcommand
- Branch and file with the same name: `git checkout main` is ambiguous. Use `--`
- A missing trailing `/` in `gitdir:~/work/`. Without it the pattern won't match subfolders

✓ TIPS & BEST PRACTICES

- Keep aliases readable: `git config --get-regexp ^alias` lists them
- Use `git -c core.pager=cat log` for a one-off setting
- Read `git help revisions` once. It explains `HEAD~`, `@{u}`, and ranges

QUICK SUMMARY

- `--show-origin` — find where a value is set
- `includeIf` — per-folder identities
- `alias.x` — shortcuts; `!` for shell
- `--` — separate revisions from paths

Init & Clone

CREATING A REPOSITORY

There are two ways to get a repository: `git init` turns an existing folder into a new repo, and `git clone` copies an existing repo with its **entire history** and sets up `origin` as the remote. A **bare** repository has no working tree and is what servers store. Shallow and partial clones trade history for speed on large projects.

NOTES

- `git clone` checks out the remote's default branch
- The clone's remote is named `origin` unless you pass `-o`
- `git init` is safe to run on an existing repo; it changes nothing
- Bare repo folders end in `.git` by convention

INIT AND CLONE VARIANTS

```
git init                          # turn the current folder into a repo
git init -b main my-project       # new folder, initial branch "main"
git init --bare server.git        # bare repo (no working tree)

git clone https://github.com/user/app.git          # into ./app
git clone git@github.com:user/app.git my-app       # SSH, custom folder
git clone --branch v2.1 <url>                      # check out a tag/branch
git clone --depth 1 <url>                          # latest commit only
git clone --single-branch --branch main <url>      # one branch only
git clone --recurse-submodules <url>               # also init submodules
git clone --filter=blob:none <url>                 # partial: fetch blobs on demand

git remote add origin git@github.com:user/app.git  # link an init'd repo
git push -u origin main                            # first push
```

CODE EXPLAINED

- → `-b main` — sets the initial branch name
- → `--bare` — repo with no checkout, used as a shared remote
- → `--branch` — check out a branch or tag instead of the default
- → `--depth 1` — shallow: no older history (fast CI clones)
- → `--filter=blob:none` — full history, file contents fetched as needed
- → `remote add` + `push -u` — publish a repo you started with `init`

REMOTE URL FORMATS

URL	PROTOCOL
`https://host/user/repo.git`	HTTPS: token or credential helper
`git@host:user/repo.git`	SSH (scp-style): SSH key
`ssh://git@host:2222/repo.git`	SSH with a custom port
`file:///srv/repo.git`	local path

INSIDE .GIT/

PATH	HOLDS
`HEAD`	current branch ref
`config`	repo-level settings
`objects/`	blobs, trees, commits, packs
`refs/heads/`	local branches
`refs/tags/`	tags
`hooks/`	hook scripts (Sheet 33)

⚠ COMMON MISTAKES

- Running `git init` inside another repo. You get a nested repo the outer one can't track
- Cloning into a folder that already has files. Clone needs an empty or new folder
- Using `--depth 1` for development. Log, blame, and rebase lose history (`git fetch --unshallow` restores it)

✓ TIPS & BEST PRACTICES

- Prefer SSH URLs once your key is set up: no token prompts
- Use `--filter=blob:none` on huge repos instead of a shallow clone
- Clone with `-o upstream` when the source is someone else's repo you'll fork

QUICK SUMMARY

- `git init` — start a new repo
- `git clone <url>` — copy repo + history
- `--depth 1` — shallow clone
- `--bare` — server-side repo

Sheet 05

Ignoring Files & .gitattributes

.gitignore patterns · ! negation · core.excludesFile · git check-ignore -v · git rm --cached · .gitattributes · text=auto / eol=lf · binary

CONTROLLING WHAT GIT TRACKS

A **.gitignore** file lists patterns for untracked files Git should never offer to add: build output, dependencies, secrets, OS junk. It does **not** affect files already tracked. **.gitattributes** sets per-path behavior: line-ending normalization, binary handling, diff drivers, and merge strategies. Commit both files so the whole team shares them.

NOTES

- Patterns are relative to the `.gitignore` file's folder
- Personal ignores go in `.git/info/exclude` (not committed)
- Global ignores: `core.excludesFile ~/.gitignore_global`
- Git never tracks empty folders; add a `.gitkeep` file

.GITIGNORE AND .GITATTRIBUTES

```
# .gitignore
node_modules/        # any folder named node_modules
/build               # only build at the repo root
*.log                # every .log file
!keep.log            # ...except this one
.env*                # .env, .env.local
**/tmp/              # tmp/ at any depth
.DS_Store

# .gitattributes
* text=auto          # normalize line endings to LF in the repo
*.sh text eol=lf     # always LF on checkout
*.bat text eol=crlf  # always CRLF on checkout
*.png binary         # no diff, no EOL conversion
dist/** export-ignore   # leave out of git archive

git check-ignore -v build/app.js   # which rule matched?
git rm --cached .env               # stop tracking, keep the file
```

CODE EXPLAINED

- → `dir/` — trailing slash matches folders only
- → `/build` — leading slash anchors to this folder
- → `!` — re-include something an earlier rule ignored
- → `text=auto` — Git decides text vs binary and normalizes EOL
- → `check-ignore -v` — prints file, line, and pattern that matched
- → `rm --cached` — untrack a file that was committed before being ignored

PATTERN SYNTAX

PATTERN	MATCHES
`*.log`	any `.log` file at any depth
`doc/*.txt`	`doc/a.txt`, not `doc/x/a.txt`
`doc/**/*.pdf`	PDFs anywhere under `doc/`
`debug?.log`	`debug1.log`, `debugA.log`
`[Bb]in/`	`bin/` or `Bin/`
`\#file`	a name starting with #

COMMON ATTRIBUTES

ATTRIBUTE	EFFECT
`binary`	same as `-diff -merge -text`
`-diff`	show "binary files differ"
`diff=python`	language-aware hunk headers
`merge=ours`	keep our version (needs driver config)
`linguist-generated`	GitHub hides it in diffs
`filter=lfs`	stored by Git LFS (Sheet 32)

⚠ COMMON MISTAKES

- Adding a file to `.gitignore` after committing it. It stays tracked until you `git rm --cached` it
- `!file` inside an ignored folder. Git can't re-include a file whose parent folder is excluded
- Inline comments after a pattern. `#` only starts a comment at the beginning of a line

✓ TIPS & BEST PRACTICES

- Start from github.com/github/gitignore templates for your stack
- After adding `.gitattributes`, run `git add --renormalize .` and commit
- List ignored files with `git status --ignored`

QUICK SUMMARY

- `.gitignore` — untracked files to skip
- `check-ignore -v` — debug a rule
- `rm --cached` — untrack, keep on disk
- `text=auto` / `eol=lf` — line endings

Status & Staging

BUILDING THE NEXT COMMIT

The **staging area** lets you choose exactly what goes into the next commit. `git status` shows which changes are staged, which are not, and which files are untracked. `git add` stages whole files or, with –p, individual hunks, so one messy working tree can become several clean commits.

NOTES

- `git add` stages content as it is now; edit again and you must re-add
- `git add .` stages the current folder and below, including deletions
- `git rm` and `git mv` stage removals and renames for you
- Git detects renames by content similarity, not by `mv`

STATUS AND STAGING

```
git status                  # long form with hints
git status -sb              # short form + branch/ahead/behind
## main...origin/main [ahead 1]
 M src/app.js               # modified, not staged
M  README.md                # modified and staged
?? notes.txt                # untracked

git add README.md           # stage one file
git add src/                # stage a folder
git add -A                  # stage everything in the repo (new, changed, deleted)
git add -u                  # stage changed + deleted tracked files only
git add -p                  # pick hunks interactively
git add -N new.js           # intent-to-add: shows in git diff

git restore --staged README.md   # unstage (keep edits)
git rm old.js                    # delete + stage the deletion
git mv a.js b.js                 # rename + stage
```

CODE EXPLAINED

- → `-sb` — short format plus a branch line with ahead/behind counts
- → `XY` — X = staged column, Y = working-tree column
- → `-A` vs `-u` — `-u` skips untracked files
- → `-p` — prompts `y/n/s/e/q` per hunk
- → `restore --staged` — modern unstage (old: `git reset <file>`)
- → `git rm --cached` — unstage and untrack, but keep the file

SHORT STATUS CODES

XY	MEANING
`??`	untracked
`M_ (M + space)`	staged modification
`_M (space + M)`	unstaged modification
`MM`	staged, then edited again
`A_`	new file staged
`D_ / _D`	deleted (staged / unstaged)
`R_`	renamed (staged)
`UU`	merge conflict

ADD -P KEYS

KEY	ACTION
`y / n`	stage / skip this hunk
`s`	split into smaller hunks
`e`	edit the hunk by hand
`a / d`	stage / skip rest of file
`q`	quit
`?`	help

⚠ COMMON MISTAKES

- Committing with `-a` and expecting new files to be included. `-a` only stages tracked files
- Editing a file after `git add` and committing. The commit has the older staged version
- `git add .` from a subfolder. It only stages that subfolder (use `-A` for the whole repo)

✓ TIPS & BEST PRACTICES

- Alias `git st` to `status -sb` for a compact view
- Review what's staged with `git diff --staged` before committing
- Use `git add -p` to split unrelated changes into separate commits

QUICK SUMMARY

- `git status -sb` — compact state
- `git add -p` — stage hunks
- `git restore --staged` — unstage
- `git rm` / `git mv` — staged delete / rename

Committing

git commit -m · -a · -v · --amend · --no-edit · multi-line messages · Conventional Commits · --allow-empty

RECORDING SNAPSHOTS

A commit saves the staged snapshot with a message, author, timestamp, and a link to its parent. Good commits are **small and focused**, and good messages explain **why**, not just what. `--amend` replaces the last commit, which is handy for typos but rewrites history, so only amend commits you haven't pushed.

NOTES

- Commits are immutable; amend creates a new commit with a new hash
- Message convention: 50-char subject, blank line, body wrapped at 72
- Write the subject in the imperative: "Fix login bug"
- `-a` skips staging for tracked files only

COMMIT VARIANTS

```
git commit                          # opens the editor for the message
git commit -m "Add login form"      # message inline
git commit -m "Fix crash" -m "Null user caused a crash on logout."
git commit -am "Update styles"      # stage tracked changes + commit
git commit -v                       # show the diff in the editor

git commit --amend                  # edit the last message (and add staged changes)
git commit --amend --no-edit        # add staged changes, keep the message
git commit --amend --reset-author # fix author after changing user.email
git commit --allow-empty -m "Trigger CI"
git commit --fixup=a1b2c3d          # mark as a fix for an older commit (Sheet 19)

git commit -S -m "Signed"           # sign the commit (Sheet 23)
```

CODE EXPLAINED

- → `-m` twice — first is the subject, second is the body
- → `-a` — auto-stages modified and deleted tracked files
- → `-v` — includes the diff below the message template
- → `--amend --no-edit` — fold forgotten changes into the last commit
- → `--allow-empty` — a commit with no changes
- → `--fixup` — creates a `fixup! …` commit for `rebase --autosquash`

CONVENTIONAL COMMITS

PREFIX	USE FOR
`feat:`	a new feature
`fix:`	a bug fix
`docs:`	documentation only
`refactor:`	code change, no behavior change
`test:`	adding or fixing tests
`chore:`	build, deps, tooling
`feat!:`	breaking change

MESSAGE ANATOMY

PART	GUIDELINE
`Subject`	≤ 50 chars, imperative, no period
`Blank line`	required between subject and body
`Body`	why the change was needed; wrap at 72
`Trailer`	`Co-authored-by: Name <email>`
`Trailer`	`Fixes #123` closes the issue

⚠ COMMON MISTAKES

- Amending a commit you've already pushed. Others now have a different history; you'll need a force push
- Giant "WIP" commits. They make `revert`, `bisect`, and review much harder
- Messages like "fix" or "update". Future you can't tell what changed or why

✓ TIPS & BEST PRACTICES

- Commit early and often locally; tidy up with interactive rebase before pushing
- Use `git commit -v` to re-read your diff while writing the message
- Add `Co-authored-by:` trailers when pairing

QUICK SUMMARY

- `commit -m` — commit staged changes
- `commit --amend` — replace the last commit
- `--no-edit` — keep the message
- `--fixup` — prepare an autosquash fix

Viewing Diffs

COMPARING SNAPSHOTS

`git diff` compares any two of: the working tree, the staging area, and any commit. With no arguments it shows **unstaged** changes only, which surprises many people. Add `--staged` for what will be committed, or name commits and branches to compare history. Options control the output format, whitespace handling, and which files are included.

NOTES

- `--staged` and `--cached` are the same option
- `A...B` in diff means: changes on B since it branched from A
- Use `-- <path>` to limit any diff to files or folders
- `diff.algorithm histogram` often gives cleaner diffs

DIFF VARIANTS

```
git diff                          # working tree vs staging area (unstaged)
git diff --staged                 # staging area vs HEAD (what you'll commit)
git diff HEAD                     # working tree vs HEAD (all changes)
git diff HEAD~3                   # working tree vs 3 commits ago
git diff main feature             # tip of main vs tip of feature
git diff main...feature           # feature's changes since it left main
git diff a1b2c3d b4e5f6a -- src/ # two commits, src/ only

git diff --stat                   # files changed + insertions/deletions
git diff --name-only main         # just file names
git diff --name-status main       # M / A / D / R per file
git diff --word-diff              # inline word changes (prose, docs)
git diff -w                       # ignore whitespace
git diff --color-moved            # highlight moved lines
git difftool -t vscode            # open in a GUI diff tool
```

CODE EXPLAINED

- → no args — only what you haven't staged yet
- → `--staged` — the exact content of your next commit
- → `main...feature` — diffs from the merge base: ideal for PR review
- → `--stat` / `--name-only` — summaries instead of full patches
- → `--word-diff` — shows `[-old-]{+new+}` inline
- → `-w` — hides indentation-only noise

WHAT GETS COMPARED

COMMAND	LEFT → RIGHT
`git diff`	index → working tree
`git diff --staged`	HEAD → index
`git diff HEAD`	HEAD → working tree
`git diff A B`	commit A → commit B
`git diff A...B`	merge-base(A,B) → B

READING A PATCH

LINE	MEANING
`--- a/file`	old version
`+++ b/file`	new version
`@@ -10,4 +10,6 @@`	old start,count → new start,count
`-line`	removed
`+line`	added

⚠ COMMON MISTAKES

- Seeing an empty `git diff` after `git add` and thinking nothing changed. Use `--staged`
- Using `A..B` in diff expecting the log meaning. In diff, `A..B` equals `A B`
- Reviewing a branch with `git diff main feature`. It also shows main's newer changes reversed; use `...`

✓ TIPS & BEST PRACTICES

- Run `git diff --staged` right before every commit
- Set `git config --global diff.colorMoved zebra` to spot moved code
- Use `git diff --check` to catch trailing whitespace and conflict markers

QUICK SUMMARY

- `git diff` — unstaged changes
- `--staged` — staged changes
- `A...B` — branch changes since fork
- `--stat` — summary view

Discarding Working Changes

THROWING AWAY LOCAL EDITS

`git restore` replaces files in your working tree (or staging area) with a known version, and `git clean` deletes **untracked** files. Both are **destructive**: uncommitted work they remove is not in the reflog and generally can't be recovered. Preview first, and stash if you're unsure (Sheet 27).

NOTES

- `restore` arrived in Git 2.23 to replace file-mode `checkout`
- `restore` with no flags = `--worktree` from the index
- `clean` refuses to run without `-f` (or `-n` / `-i`)
- Ignored files are only removed with `-x` or `-X`

RESTORE AND CLEAN

```
git restore app.js                      # discard unstaged edits to a file
git restore .                           # discard all unstaged edits
git restore -p app.js                   # discard hunk by hunk
git restore --staged app.js             # unstage only (edits kept)
git restore --staged --worktree app.js  # unstage AND discard edits
git restore --source=HEAD~2 app.js      # bring back an older version
git restore --source=main -- docs/      # take a folder from another branch

git checkout -- app.js                  # legacy form of: git restore app.js

git clean -n                            # dry run: what would be deleted
git clean -f                            # delete untracked files
git clean -fd                           # ...and untracked folders
git clean -fdx                          # ...and ignored files (build output)
git clean -i                            # interactive menu
```

CODE EXPLAINED

- → `restore <file>` — copy the staged version over your edits
- → `--staged` — reset the index entry to HEAD
- → `--source=<rev>` — restore from any commit, branch, or tag
- → `-p` — choose which hunks to throw away
- → `clean -n` — always run first; it only lists
- → `-x` — also removes ignored files like `node_modules/`

WHICH COMMAND?

GOAL	COMMAND
Discard edits in a file	`git restore <file>`
Unstage a file	`git restore --staged <file>`
File from an old commit	`git restore --source=<sha> <file>`
Delete untracked files	`git clean -fd`
Reset everything to HEAD	`git reset --hard` (Sheet 25)
Keep changes for later	`git stash -u` (Sheet 27)

GIT CLEAN FLAGS

FLAG	EFFECT
`-n`	dry run
`-f`	force (required)
`-d`	include untracked folders
`-x`	include ignored files too
`-X`	only ignored files
`-e <pattern>`	exclude a pattern

⚠ COMMON MISTAKES

- Running `git clean -fdx` without `-n` first. It deletes `.env` files and local config too
- Expecting `git restore` to undo a commit. It only touches files; use `revert` or `reset`
- Restoring and losing work you wanted. Uncommitted content isn't in the reflog; stash when unsure

✓ TIPS & BEST PRACTICES

- Use `git stash -u` as a reversible alternative to `restore` + `clean`
- Add `-p` to `restore` to keep the good hunks
- Use `git clean -fdX` to wipe only build artifacts

QUICK SUMMARY

- `restore <file>` — discard edits
- `restore --staged` — unstage
- `--source=<rev>` — older version
- `clean -n` then `-fd` — remove untracked

Log Basics

BROWSING HISTORY

`git log` lists commits reachable from HEAD, newest first. On its own it is verbose, so most people use a compact alias built from `--oneline --graph --all`. Add `-p` to see each commit's patch or `--stat` for a per-file summary. Give it a path to see only commits that touched that file.

NOTES

- Press `q` to leave the pager, `/` to search inside it
- `--all` includes every branch and tag, not just HEAD
- `--decorate` shows branch/tag names (on by default in a terminal)
- `--follow` tracks a single file across renames

EVERYDAY LOG COMMANDS

```
git log                              # full log: hash, author, date, message
git log --oneline                    # a1b2c3d Fix login redirect
git log -n 5                         # last 5 commits (also: -5)
git log --oneline --graph --all      # ASCII branch graph of everything
git log -p                           # each commit with its diff
git log --stat                       # files changed per commit
git log main..feature                # commits on feature not on main
git log -- src/auth.js               # commits that touched a file
git log --follow -- src/login.js     # ...including before a rename
git log -L 40,60:src/app.js          # history of lines 40-60

git shortlog -sn                     # commit count per author
git shortlog -sn --since="1 month ago"
git log --oneline | wc -l            # number of commits on HEAD
```

CODE EXPLAINED

- → `--oneline` — short hash + subject, one line per commit
- → `--graph` — draws branch and merge lines
- → `main..feature` — reachable from feature, not from main
- → `-- <path>` — only commits that changed that path
- → `-L start,end:file` — line-range history with diffs
- → `shortlog -sn` — summary, numbered, sorted by count

COMMON OPTIONS

OPTION	EFFECT
`--oneline`	compact one-line view
`--graph`	branch/merge graph
`--all`	all refs, not just HEAD
`-p / --patch`	show diffs
`--stat`	per-file change summary
`--reverse`	oldest first
`--first-parent`	only the mainline of merges

HANDY ALIASES

ALIAS	EXPANDS TO
`lg`	`log --oneline --graph --all --decorate`
`last`	`log -1 --stat`
`hist`	`log --pretty=format:'%h %ad %s' --date=short`
`who`	`shortlog -sn --no-merges`

⚠ COMMON MISTAKES

- Forgetting `--all` and thinking a branch's commits are gone. Plain `git log` only follows HEAD
- Using `--follow` with multiple paths. It only works for a single file
- Reading a busy graph without `--oneline`. Combine them or the graph is unreadable

✓ TIPS & BEST PRACTICES

- Set `git config --global alias.lg "log --oneline --graph --all"`
- Use `git log --first-parent main` to see just merged PRs
- Pipe to `--no-pager` when scripting: `git --no-pager log -5`

QUICK SUMMARY

- `log --oneline` — compact history
- `--graph --all` — visual branch map
- `-p` / `--stat` — details per commit
- `-- <file>` / `--follow` — file history

Log Filtering & Formatting

FINDING THE RIGHT COMMIT

Filters narrow the log by author, date, message, path, or by the **content** a commit added or removed. The pickaxe options –S and –G are the fastest way to find when a function or string appeared or disappeared. `--pretty=format:` builds custom output for reports, changelogs, and scripts.

NOTES

- Multiple `--author` or `--grep` filters are OR'd by default
- Add `--all-match` to require every `--grep` to match
- Dates accept `2026-01-31`, `"2 weeks ago"`, `yesterday`
- `-i` makes `--grep` and `--author` case-insensitive

FILTERS AND FORMATS

```
git log --author="Ada"                    # commits by an author (regex)
git log --since="2 weeks ago" --until=yesterday
git log --grep="JIRA-142"                 # search commit messages
git log -S "parseToken" --oneline         # added/removed this string
git log -G "timeout\s*=" --oneline        # diff lines matching a regex
git log --no-merges main..feature         # skip merge commits
git log --merges --first-parent main      # only merge commits on main
git log --diff-filter=D --name-only       # commits that deleted files

git log --pretty=format:"%h %ad %an %s" --date=short
# a1b2c3d 2026-10-02 Ada Lovelace Fix login redirect
git log --pretty=format:"- %s (%h)" v1.2..v1.3   # changelog lines
git log --format="%H" -1                  # full hash of HEAD (scripts)
```

CODE EXPLAINED

- → `--author` — matches name or email, regex allowed
- → `-S <string>` — commits where the string's count changed
- → `-G <regex>` — commits whose diff has a matching line
- → `--diff-filter=D` — A added, M modified, D deleted, R renamed
- → `--pretty=format:` — custom output using placeholders
- → `--date=short` — `YYYY-MM-DD`; also `relative`, `iso`

FORMAT PLACEHOLDERS

CODE	PRINTS
`%H / %h`	full / short hash
`%an / %ae`	author name / email
`%ad / %ar`	author date / relative date
`%s`	subject line
`%b`	body
`%d`	ref names (branches, tags)
`%C(yellow)…%Creset`	color

FILTER CHEAT LIST

OPTION	FINDS
`--author=<re>`	by author
`--committer=<re>`	by committer
`--grep=<re>`	by message
`-S<str>`	string added/removed
`-G<re>`	regex in diff
`--since / --until`	date range
`-- <path>`	touching a path

⚠ COMMON MISTAKES

- Using `--grep` to find code. It searches messages only; use `-S` or `-G` for content
- Quoting regex characters in `--author`. `--author="J. Smith"` treats `.` as any character
- Forgetting `--date=` in a format. `%ad` then prints the long default date

✓ TIPS & BEST PRACTICES

- Add `-p` to `-S` searches to see the exact change
- Use `%C(auto)%d` in formats to get colored branch names
- Combine `--since` and `shortlog -sn` for team activity reports

QUICK SUMMARY

- `--author` / `--grep` — who / message
- `-S` / `-G` — content search
- `--since` — date range
- `--pretty=format:` — custom output

Referencing Commits

HEAD~2 vs HEAD^2 · short SHAs · @{u} upstream · main@{yesterday} · A..B / A...B · <sha>:path · git rev-parse

REVISION SYNTAX

Almost every Git command accepts a **revision**: a hash, branch, tag, or an expression relative to one. ~ walks back through first parents, ^ picks a specific parent of a merge, @{…} reads the reflog or upstream, and `..` / `...` describe ranges. Learn these once and they work in `log`, `diff`, `show`, `reset`, and `rebase`.

NOTES

- `@` alone is shorthand for `HEAD`
- `HEAD~1` and `HEAD^` are the same commit
- Quote braces in some shells: `'HEAD@{1}'`
- `git help revisions` has the full grammar

REVISION EXPRESSIONS

```
git show HEAD~2                # grandparent (follow first parents)
git show HEAD^2                # second parent of a merge commit
git show a1b2c3d               # any unique hash prefix (4+ chars)
git show v1.4.0^{commit}       # the commit an annotated tag points to
git show main:src/app.js       # a file as it is on main
git diff @{u}                  # working tree vs upstream (origin/main)
git log @{u}..                 # local commits not yet pushed
git log ..@{u}                 # upstream commits you haven't pulled
git show main@{yesterday}      # where main pointed yesterday (reflog)
git switch @{-1}               # previous branch (same as: git switch -)
git log --left-right --oneline main...feature   # < main  > feature

git rev-parse HEAD             # full hash
git rev-parse --short HEAD     # a1b2c3d
git rev-parse --abbrev-ref HEAD   # current branch name
```

CODE EXPLAINED

- → `~n` — n steps back along first parents
- → `^n` — the nth parent of a merge commit
- → `rev:path` — a blob or tree at that revision
- → `@{u}` — the branch's upstream tracking branch
- → `@{-1}` — the branch you were on before
- → `rev-parse` — turns any revision into a hash or name

RELATIVE REFS

SYNTAX	MEANING
`HEAD~3`	3 commits back
`HEAD^ / HEAD^1`	first parent
`HEAD^2`	second parent (merges)
`HEAD~2^2`	2nd parent of grandparent
`@{u} / @{push}`	upstream / push target
`@{-1}`	previous branch

RANGES

SYNTAX	IN GIT LOG
`A..B`	in B, not in A
`A...B`	in A or B, not both
`^A B`	same as `A..B`
`B --not A C`	in B, not in A or C
`A^!`	just commit A

REFLOG REFS

SYNTAX	MEANING
`HEAD@{2}`	HEAD two moves ago
`main@{1}`	main's previous value
`main@{1.hour.ago}`	main an hour ago
`@{0}`	current branch now

⚠ COMMON MISTAKES

- Mixing up `~` and `^`. `HEAD^2` is a merge's second parent, not two commits back
- Expecting `..` to mean the same in `diff` and `log`. In diff, `A..B` is just `A B`
- Using `@{yesterday}` on a fresh clone. The reflog is local and starts at clone time

✓ TIPS & BEST PRACTICES

- Use `git log @{u}..` before pushing to see what will go up
- In scripts, prefer `git rev-parse --verify <rev>` to validate input
- Use `git show HEAD:path > old.js` to save an old version of a file

QUICK SUMMARY

- `~` — walk back first parents
- `^n` — choose a merge parent
- `A..B` / `A...B` — ranges
- `@{u}` / `@{-1}` — upstream / last branch

Show, Blame & Grep

INSPECTING CODE AND COMMITS

`git show` displays any object: a commit with its diff, a tag, or a file at a revision. `git blame` labels each line with the commit that last changed it, which is the quickest route from "why is this here?" to the commit message and PR. `git grep` searches tracked files fast, at any revision, and `git describe` names a commit relative to the nearest tag.

NOTES

- Blame shows the last change, not necessarily the author of the idea
- `-w` ignores whitespace-only changes in blame
- `.git-blame-ignore-revs` skips formatting-only commits
- `git grep` skips untracked and ignored files

SHOW, BLAME, GREP, DESCRIBE

```
git show                          # HEAD: message + diff
git show a1b2c3d --stat           # a commit's file summary
git show v2.0                     # tag info + the commit it points to
git show HEAD~3:src/app.js        # file contents at a revision

git blame src/app.js              # who changed each line, and when
git blame -L 40,60 src/app.js     # only lines 40-60
git blame -L :login src/app.js    # only the function "login"
git blame -w -C src/app.js        # ignore whitespace, detect moved code
git blame --ignore-revs-file .git-blame-ignore-revs src/app.js
git config blame.ignoreRevsFile .git-blame-ignore-revs   # make it default

git grep -n "TODO"                # search tracked files with line numbers
git grep -c "useState" -- '*.tsx' # count matches per file
git grep "apiUrl" v1.0            # search an older revision
git describe --tags               # v1.4.0-12-ga1b2c3d
```

CODE EXPLAINED

- → `show rev:path` — prints the file, no diff
- → `blame -L a,b` — limit to a line range or `:funcname`
- → `-C` — detect lines moved or copied from other files
- → `--ignore-revs-file` — skip listed bulk-format commits
- → `git grep <rev>` — search without checking it out
- → `describe` — tag, commits since tag, `g` + short hash

GIT GREP OPTIONS

OPTION	EFFECT
`-n`	line numbers
`-i`	case-insensitive
`-w`	whole words only
`-l`	file names only
`-c`	count per file
`-E`	extended regex
`--untracked`	include untracked files

DESCRIBE OUTPUT

OUTPUT	MEANS
`v1.4.0`	HEAD is exactly the tag
`v1.4.0-12-ga1b2c3d`	12 commits after v1.4.0
`--tags`	also use lightweight tags
`--dirty`	append `-dirty` if modified
`--always`	fall back to a short hash

⚠ COMMON MISTAKES

- Blaming the person on a reformatted line. A formatter commit hides the real author; use `-w` or ignore-revs
- Using `grep -r` in a repo. It also searches `.git/` and `node_modules/`; `git grep` doesn't
- Expecting `describe` to find lightweight tags. It needs `--tags` for those

✓ TIPS & BEST PRACTICES

- Click through blame: `git show <sha>` on the hash it prints
- Commit a `.git-blame-ignore-revs` file; GitHub honors it too
- Use `git describe --tags --always --dirty` for build version strings

QUICK SUMMARY

- `git show` — inspect any object
- `git blame -L` — line history
- `git grep` — fast tracked-file search
- `git describe` — tag-based name

Sheet 14

Creating & Managing Branches

git branch -a / -vv · switch / switch -c / switch - · checkout -b (legacy) · branch -m · -d / -D · --merged / --contains · --sort=-committerdate · detached HEAD

BRANCHES ARE POINTERS

A branch is just a **movable pointer** to a commit; committing moves the current branch forward. `git switch` (Git 2.23+) changes branches, while `git branch` lists, renames, and deletes them. Checking out a commit or tag directly puts you in **detached HEAD**: fine for looking around, but create a branch before you commit.

NOTES

- `switch` refuses to discard uncommitted changes that conflict
- `origin/main` is a remote-tracking branch; you don't commit to it
- `-d` only deletes merged branches; `-D` forces
- Naming: `feature/login`, `fix/123-crash`, lowercase, hyphens

BRANCH COMMANDS

```
git branch                          # local branches (* = current)
git branch -a                       # local + remote-tracking
git branch -vv                      # last commit + upstream + ahead/behind
git switch -c feature/login         # create and switch
git switch -c hotfix v1.4.0         # new branch from a tag/commit
git switch main                     # switch to an existing branch
git switch -                        # back to the previous branch
git switch feature/x                # auto-tracks origin/feature/x if it exists
git checkout -b feature/login       # legacy equivalent of switch -c

git branch -m old-name new-name     # rename (-m alone renames current)
git branch -d feature/login         # delete if merged
git branch -D spike/test            # force delete
git branch --merged main            # safe to delete
git branch --contains a1b2c3d       # which branches have this commit
git branch --sort=-committerdate    # most recently active first
git switch --detach v1.4.0          # explore a tag (detached HEAD)
```

CODE EXPLAINED

- → `-vv` — shows `[origin/main: ahead 2]` style tracking info
- → `switch -c <name> <start>` — branch from any revision
- → `switch -` — toggle between two branches
- → `-m` — rename; `-M` forces over an existing name
- → `--merged` / `--no-merged` — cleanup helpers
- → `--detach` — look around without moving a branch

LEGACY VS MODERN

OLD (CHECKOUT)	NEW
`git checkout dev`	`git switch dev`
`git checkout -b dev`	`git switch -c dev`
`git checkout -`	`git switch -`
`git checkout <sha>`	`git switch --detach <sha>`
`git checkout -- f`	`git restore f`

DETACHED HEAD

SITUATION	DO THIS
Just looking	`git switch -` to go back
Made commits to keep	`git switch -c new-branch`
Already left them	find via `git reflog`, then branch

⚠ COMMON MISTAKES

- Committing in detached HEAD and switching away. The commits are only reachable from the reflog
- Deleting with `-D` to silence a warning. `-d` failing means the work isn't merged anywhere
- Renaming a pushed branch locally only. Push the new name and delete the old one on the remote

✓ TIPS & BEST PRACTICES

- Set `git config --global branch.sort -committerdate`
- Clean up merged branches: `git branch --merged main | grep -v main`
- Use `git switch -c` from the branch you want as the base

QUICK SUMMARY

- `switch -c` — create + switch
- `branch -vv` — tracking status
- `branch -d` / `-D` — delete
- `--detach` — inspect without a branch

Tags

MARKING RELEASES

A tag is a fixed name for a commit, usually a release like `v1.2.0`. **Annotated** tags are full objects with a tagger, date, and message (and optionally a signature), and they are what releases should use. **Lightweight** tags are just a name pointing at a commit. Tags are not pushed by default, so push them explicitly.

NOTES

- Tags don't move when new commits arrive (unlike branches)
- Semantic versioning: `vMAJOR.MINOR.PATCH`
- `git describe` and `--follow-tags` only see annotated tags by default
- Checking out a tag gives detached HEAD

CREATING, LISTING, SHARING TAGS

```
git tag v1.0.0                          # lightweight tag on HEAD
git tag -a v1.1.0 -m "Release 1.1.0"    # annotated tag on HEAD
git tag -a v1.0.1 a1b2c3d -m "Hotfix"   # tag an older commit
git tag -s v2.0.0 -m "Signed release"   # signed annotated tag

git tag                                 # list all tags
git tag -l "v1.*"                       # list matching a pattern
git tag --sort=-v:refname               # newest version first
git show v1.1.0                         # tag message + commit

git push origin v1.1.0                  # push one tag
git push --follow-tags                  # push commits + their annotated tags
git push origin --tags                  # push every tag (careful)
git fetch --tags                        # get all remote tags

git tag -d v1.0.0                       # delete locally
git push origin --delete refs/tags/v1.0.0   # delete on the remote
```

CODE EXPLAINED

- → `-a -m` — annotated tag with a message
- → `git tag <name> <sha>` — tag any commit
- → `-s` — sign with your GPG/SSH key
- → `--sort=-v:refname` — version-aware sort, descending
- → `--follow-tags` — pushes annotated tags reachable from pushed commits
- → `refs/tags/` prefix — avoids clashing with a branch of the same name

ANNOTATED VS LIGHTWEIGHT

FEATURE	ANNOTATED / LIGHTWEIGHT
Object	tag object / plain ref
Tagger + date	yes / no
Message	yes / no
Signable	yes / no
Use for	releases / private bookmarks

SEMANTIC VERSIONING

BUMP	WHEN
MAJOR (2.0.0)	breaking API change
MINOR (1.3.0)	new feature, compatible
PATCH (1.2.4)	bug fix only
1.3.0-rc.1	pre-release

⚠ COMMON MISTAKES

- Expecting `git push` to send tags. Push them by name or with `--follow-tags`
- Moving a published tag. Others keep the old one; create a new version instead
- `git push --tags` with stray local tags. It publishes every experimental tag too

✓ TIPS & BEST PRACTICES

- Set `git config --global push.followTags true`
- Create GitHub releases from tags: `gh release create v1.1.0`
- Verify signed tags with `git tag -v v2.0.0`

QUICK SUMMARY

- `tag -a -m` — annotated release tag
- `tag -l` — list / filter
- `push --follow-tags` — share tags
- `tag -d` + `push --delete` — remove

Merging

COMBINING BRANCHES

`git merge` brings another branch's commits into the current branch. If your branch hasn't moved, Git just slides the pointer forward (**fast-forward**). Otherwise it creates a **merge commit** with two parents. Options let you force a merge commit, refuse anything but a fast-forward, or squash everything into one staged change.

NOTES

- Always merge *into* the branch you're on: switch first
- A clean working tree avoids most merge errors
- Merge commits preserve the exact history of both branches
- Conflicts pause the merge; see Sheet 17

MERGE VARIANTS

```
git switch main
git merge feature/login             # fast-forward if possible, else merge commit
git merge --no-ff feature/login     # always create a merge commit
git merge --ff-only origin/main     # only fast-forward; fail otherwise
git merge --squash feature/login    # stage all changes, no commit yet
git commit -m "Add login (#42)"     # ...then commit the squash yourself
git merge -m "Merge release 2.1" release/2.1

git merge -X theirs feature         # on conflicting hunks, prefer theirs
git merge -X ours feature           # on conflicting hunks, prefer ours
git merge --abort                   # stop and return to the pre-merge state
git merge --continue                # finish after resolving conflicts

git log --oneline --graph -5        # see the merge shape
git branch --merged                 # branches fully merged into HEAD
```

CODE EXPLAINED

- → fast-forward — no new commit; the branch pointer just moves
- → `--no-ff` — keeps a visible merge bubble for the feature
- → `--ff-only` — safe sync; errors if histories diverged
- → `--squash` — one combined change, no link to the branch history
- → `-X ours|theirs` — strategy option for conflicting hunks only
- → `--abort` — restores the branch and working tree

MERGE RESULTS

TYPE	HISTORY LOOKS LIKE
`Fast-forward`	straight line, no merge commit
`Merge commit`	two-parent commit joins lines
`Squash merge`	one new commit, branch not linked
`Rebase + ff`	straight line, original commits replayed

STRATEGY OPTIONS

OPTION	EFFECT
`-X ours`	conflicts resolved to our side
`-X theirs`	conflicts resolved to their side
`-s ours`	ignore their changes entirely
`-X ignore-space-change`	whitespace-tolerant
`--allow-unrelated-histories`	join two separate repos

⚠ COMMON MISTAKES

- Confusing `-X ours` with `-s ours`. `-s ours` discards *all* of the other branch's changes
- Squash-merging, then continuing to use the branch. Git doesn't know it was merged, so you get repeat conflicts
- Merging with uncommitted changes. If the merge fails they can get tangled; commit or stash first

✓ TIPS & BEST PRACTICES

- Set `git config --global merge.ff only` for a no-surprises `git merge`
- Preview with `git log --oneline main..feature` before merging
- Undo a just-finished merge with `git reset --hard ORIG_HEAD`

QUICK SUMMARY

- `merge <branch>` — combine into current
- `--no-ff` / `--ff-only` — control the shape
- `--squash` — one commit
- `--abort` — back out

Resolving Conflicts

WHEN GIT CAN'T DECIDE

A conflict happens when both sides changed the same lines (or one side edited a file the other deleted). Git pauses, writes **conflict markers** into the file, and marks it as unmerged. You edit the file to the correct result, `git add` it, and continue. The same steps apply during `merge`, `rebase`, `cherry-pick`, `revert`, and `stash pop`.

NOTES

- During a **rebase**, `ours` is the branch you're rebasing onto
- `zdiff3` style also shows the common ancestor's version
- `rerere` replays resolutions you've made before
- `git diff` during a conflict shows a combined diff

CONFLICT WORKFLOW

```
git merge feature            # CONFLICT (content): Merge conflict in app.js
git status                   # "both modified: app.js"
git diff --name-only --diff-filter=U   # list conflicted files

# app.js with merge.conflictStyle=zdiff3
<<<<<<< HEAD
const timeout = 5000;
||||||| base
const timeout = 3000;
=======
const timeout = 10000;
>>>>>>> feature

git checkout --ours   app.js   # take our whole version of the file
git checkout --theirs app.js   # take their whole version
git mergetool                  # open the configured merge tool
git add app.js                 # mark resolved
git merge --continue           # or: git commit / git rebase --continue
```

CODE EXPLAINED

- → `<<<<<<< HEAD` — your side (current branch)
- → `||||||| base` — the original, shown with `zdiff3`
- → `>>>>>>> feature` — the incoming side
- → `--diff-filter=U` — only unmerged files
- → `--ours` / `--theirs` — pick one side for the whole file
- → `git add` — tells Git the file is resolved

OURS VS THEIRS

OPERATION	OURS / THEIRS
`merge X`	current branch / X
`rebase main`	main (new base) / your commit
`cherry-pick C`	current branch / C
`stash pop`	current tree / the stash

CONFLICT SETUP

CONFIG	EFFECT
`merge.conflictStyle zdiff3`	show the base
`rerere.enabled true`	reuse resolutions
`merge.tool vscode`	tool for `git mergetool`
`mergetool.keepBackup false`	no `.orig` files

GET OUT

COMMAND	EFFECT
`merge --abort`	undo the merge attempt
`rebase --abort`	undo the rebase
`cherry-pick --abort`	undo the pick
`git log --merge`	commits that touch conflicts

⚠ COMMON MISTAKES

- Committing with markers still in the file. Run `git diff --check` to find stray `<<<<<<<` lines
- Using `--ours` during a rebase to keep *your* work. In a rebase, `--theirs` is your commit
- Resolving by deleting the other side blindly. Read both versions; often you need parts of each

✓ TIPS & BEST PRACTICES

- Turn on `zdiff3` : seeing the base makes most conflicts obvious
- Run the tests before `--continue` ; a clean merge can still be broken
- Keep branches short-lived to keep conflicts small

QUICK SUMMARY

- markers — `<<<<<<<` , `=======` , `>>>>>>>`
- edit → `git add` → `--continue`
- `--ours` / `--theirs` — pick a side
- `--abort` — start over

Rebasing

REPLAYING COMMITS

`git rebase` takes your branch's commits and **replays them on top of** another base, producing a straight, linear history. The replayed commits are new commits with new hashes, which is why the golden rule is: **never rebase commits other people already have**. Rebasing your own unpushed or personal feature branch onto `main` is normal and safe.

NOTES

- Conflicts are resolved commit by commit
- `git pull --rebase` = fetch + rebase onto upstream
- After rebasing a pushed branch, push with `--force-with-lease`
- `ORIG_HEAD` holds where you were before the rebase

REBASE VARIANTS

```
git switch feature
git rebase main                  # replay feature's commits onto main
git rebase origin/main           # onto the freshly fetched remote main
git pull --rebase                # fetch + rebase onto upstream

# during a conflict
git add app.js && git rebase --continue
git rebase --skip                # drop the commit that conflicts
git rebase --abort               # return to the pre-rebase state

git rebase --onto main old-base feature   # move commits after old-base onto main
git rebase --update-refs main     # also move stacked branches in the range
git rebase -r main                # keep merge commits (--rebase-merges)

git config --global pull.rebase true
git config --global rebase.autoStash true   # stash/unstash around rebases
git reset --hard ORIG_HEAD        # undo the last rebase
```

CODE EXPLAINED

- → `rebase main` — base becomes main's tip; commits replayed in order
- → `--continue` — after resolving and staging the conflict
- → `--skip` — discards the current commit entirely
- → `--onto A B C` — take commits in `B..C` and put them on A
- → `--update-refs` — keeps stacked branches pointing at rewritten commits
- → `autoStash` — lets you rebase with a dirty working tree

REBASE VS MERGE

ASPECT	REBASE / MERGE
History	linear / branched
Commit hashes	rewritten / unchanged
Conflicts	per commit / once
Shared branches	avoid / safe
Best for	updating your branch / integrating

--ONTO EXAMPLE

BEFORE	AFTER
`main: A-B`	main: A-B
`old: A-C`	feature: A-B-E'-F'
`feature: A-C-E-F`	(C is left behind)
`cmd`	`rebase --onto main old feature`

⚠ COMMON MISTAKES

- Rebasing `main` or any shared branch. Everyone else's history now disagrees with yours
- Plain `git push --force` after a rebase. It can overwrite a teammate's push; use `--force-with-lease`
- Panicking mid-rebase. `git status` says which commit you're on; `--abort` always gets you back

✓ TIPS & BEST PRACTICES

- Rebase onto `origin/main` right after `git fetch` to stay current
- Compare before and after a rebase with `git range-diff ORIG_HEAD...HEAD`
- Enable `rerere` so repeated rebases reuse conflict fixes

QUICK SUMMARY

- `rebase main` — replay onto main
- `--continue` / `--skip` / `--abort`
- `--onto` — transplant a range
- never rebase shared history

Interactive Rebase

rebase -i HEAD~5 · pick reword edit · squash fixup drop · exec · reordering · commit --fixup + --autosquash

EDITING YOUR OWN HISTORY

Interactive rebase opens a **todo list** of commits that you edit to reorder, combine, reword, split, or drop them before sharing. It's how messy local work ("wip", "fix typo") becomes a clean series of reviewable commits. Save and close the editor to start; Git pauses where you asked it to.

NOTES

- The list is oldest at the top (opposite of `git log`)
- Deleting a line drops that commit
- `HEAD~5` edits the last 5 commits; `-i main` edits everything since main
- Same golden rule: only for commits not yet shared

A TODO LIST

```
git rebase -i HEAD~5      # or: git rebase -i main

pick   a1b2c3d Add login form
reword b2c3d4e Add validaton        # fix the typo in this message
fixup  c3d4e5f fix lint             # fold into the commit above, drop msg
squash d4e5f6a Add error messages   # fold in, combine messages
edit   e5f6a7b Add API client       # stop here to amend or split
exec   npm test                     # run a command; stop if it fails
drop   f6a7b8c debug logging        # remove this commit

# fixup workflow: mark fixes as you go, squash automatically later
git commit --fixup=a1b2c3d
git rebase -i --autosquash main
git config --global rebase.autoSquash true
```

CODE EXPLAINED

- → `reword` — stops only to edit the message
- → `fixup` — melds into the previous commit, discards this message
- → `squash` — melds in and lets you merge both messages
- → `edit` — pauses; `commit --amend` or split, then `--continue`
- → `exec` — runs a shell command between commits
- → `--autosquash` — moves `fixup!` commits under their targets

TODO COMMANDS

CMD	ACTION
`p, pick`	use the commit as-is
`r, reword`	use, but edit the message
`e, edit`	use, but stop for amending
`s, squash`	meld into previous, keep message
`f, fixup`	meld into previous, drop message
`x, exec`	run a shell command
`b, break`	stop here
`d, drop`	remove the commit

SPLIT A COMMIT (AT EDIT)

STEP	COMMAND
1	`git reset HEAD~`
2	`git add -p` (first part)
3	`git commit -m "Part 1"`
4	repeat for the rest
5	`git rebase --continue`

⚠ COMMON MISTAKES

- Reading the list top-down as newest first. The top line is the oldest commit
- Squashing the first line. `squash` / `fixup` need a commit above them to meld into
- Interactive-rebasing past a merge without `-r`. Merge commits get flattened away

✓ TIPS & BEST PRACTICES

- Use `git rebase -i --exec "npm test" main` to test every commit
- Set `sequence.editor` if you prefer a different editor for todo lists
- Escape hatch: `git rebase --abort`, or `git reset --hard ORIG_HEAD` after finishing

QUICK SUMMARY

- `rebase -i` — edit a commit list
- `fixup` / `squash` — combine
- `reword` / `edit` — change
- `--fixup` + `--autosquash` — automate

Cherry-Pick & Patches

COPYING INDIVIDUAL CHANGES

`git cherry-pick` applies the change from one or more existing commits as **new commits** on your current branch, which is the usual way to backport a fix to a release branch. Patches do the same across repos or by email: `format-patch` exports commits as files, `am` applies them as commits, and `apply` applies a plain diff without committing.

NOTES

- Cherry-picked commits get new hashes
- `-x` records the source hash in the message
- Picking a merge commit needs `-m 1` to choose the mainline
- `git apply` works on any diff, including `git diff` output

CHERRY-PICK AND PATCH COMMANDS

```
git switch release/2.1
git cherry-pick a1b2c3d              # copy one commit
git cherry-pick a1b2c3d e5f6a7b      # several commits
git cherry-pick a1b2c3d^..f6a7b8c    # a range, inclusive of a1b2c3d
git cherry-pick -x a1b2c3d           # add "(cherry picked from commit …)"
git cherry-pick -n a1b2c3d           # apply changes, don't commit
git cherry-pick --continue           # after resolving a conflict
git cherry-pick --abort              # cancel the whole pick

git format-patch -3                  # last 3 commits -> 000N-*.patch files
git format-patch main..feature -o patches/
git am patches/*.patch               # apply patch files as commits
git am --abort                       # back out of a failed am

git diff > change.diff               # plain diff
git apply --check change.diff        # test if it applies cleanly
git apply change.diff                # apply to working tree (no commit)
```

CODE EXPLAINED

- → `A^..B` — includes A; plain `A..B` would skip A
- → `-x` — traceable backports; use for public branches
- → `-n` — stage the change so you can combine or edit it
- → `format-patch` — one email-style file per commit, keeps author
- → `am` — "apply mailbox": recreates the commits
- → `apply --check` — dry run for a diff

WHICH TOOL?

GOAL	USE
Backport a fix	`cherry-pick -x`
Move commits to a new base	`rebase --onto`
Send commits by email/file	`format-patch` + `am`
Share uncommitted changes	`git diff` + `apply`
Undo a commit's change	`revert` (Sheet 26)

CHERRY-PICK FLAGS

FLAG	EFFECT
`-x`	append source hash
`-n`	no commit
`-e`	edit the message
`-m 1`	pick a merge, mainline parent 1
`--skip`	skip the current commit
`--signoff`	add Signed-off-by

⚠ COMMON MISTAKES

- Cherry-picking a whole feature commit by commit. Merge or rebase the branch instead
- Writing `A..B` and missing commit A. Use `A^..B` to include it
- Using `git apply` and expecting a commit. It only changes files; use `am` for commits

✓ TIPS & BEST PRACTICES

- Always use `-x` when picking onto a shared release branch
- If the same fix is needed in many branches, commit it on the oldest and merge forward
- `git apply -3` falls back to a 3-way merge when context has drifted

QUICK SUMMARY

- `cherry-pick <sha>` — copy a commit
- `-x` — record the source
- `format-patch` / `am` — commits as files
- `apply` — apply a diff

Remotes, Fetch & Pull

git remote -v · remote add / set-url · origin vs upstream · fetch --all --prune · pull = fetch + merge · pull --rebase · pull.ff only · origin/main tracking branches

TALKING TO OTHER REPOSITORIES

A **remote** is a named URL for another copy of the repo; `origin` is the one you cloned from. `git fetch` downloads new commits into **remote-tracking branches** like `origin/main` without touching your work. `git pull` is fetch followed by a merge (or rebase) into your current branch. Fetching first and integrating deliberately avoids surprises.

NOTES

- `origin/main` updates only when you fetch
- In forks: `origin` = your fork, `upstream` = the original
- Without `pull.rebase` or `pull.ff` set, Git asks you to choose
- `fetch.prune true` removes branches deleted on the remote

REMOTE, FETCH, PULL

```
git remote -v                              # list remotes + URLs
git remote add upstream https://github.com/org/app.git
git remote set-url origin git@github.com:me/app.git   # switch to SSH
git remote rename origin old-origin
git remote remove old-origin
git remote show origin                     # tracking + stale branch info

git fetch                                  # update origin/* refs
git fetch --all --prune                    # all remotes, drop deleted branches
git fetch upstream main                    # one branch from one remote
git log --oneline main..origin/main        # what's new upstream

git pull                                   # fetch + integrate into current branch
git pull --rebase                          # fetch + rebase local commits on top
git pull --ff-only                         # only if no local commits diverged
git config --global pull.ff only           # make --ff-only the default
git branch -u origin/main                  # set upstream for this branch
```

CODE EXPLAINED

- → `remote -v` — shows fetch and push URLs
- → `set-url` — change a remote's address (HTTPS ↔ SSH)
- → `fetch --prune` — removes stale `origin/*` refs
- → `main..origin/main` — commits you'd get by pulling
- → `pull --rebase` — linear history, no merge bubbles
- → `branch -u` — sets the upstream used by bare `pull` / `push`

PULL BEHAVIOR

SETTING	GIT PULL DOES
`pull.rebase false`	merge (may create merge commit)
`pull.rebase true`	rebase local commits
`pull.ff only`	fast-forward or fail
`--rebase / --no-rebase`	one-time override

BRANCH KINDS

REF	WHAT IT IS
`main`	your local branch
`origin/main`	last-fetched remote state
`main@{u}`	main's upstream
`refs/remotes/`	where tracking refs live

⚠ COMMON MISTAKES

- Thinking `origin/main` is live. It's a snapshot from your last fetch
- Running `git pull` with uncommitted changes. Commit or stash first, or set `rebase.autoStash`
- Pulling with merge on a feature branch repeatedly. You get a trail of meaningless merge commits

✓ TIPS & BEST PRACTICES

- Prefer `git fetch` + review + `git rebase origin/main` for full control
- Sync a fork: `git fetch upstream && git rebase upstream/main`
- Set `fetch.prune true` globally

QUICK SUMMARY

- `remote add` / `-v` — manage remotes
- `fetch` — download, don't touch work
- `pull` — fetch + merge/rebase
- `origin/main` — tracking snapshot

Push

PUBLISHING COMMITS

`git push` uploads your commits and moves the branch on the remote. The remote only accepts a **fast-forward**: if someone else pushed first, you're rejected until you integrate their work. After rewriting history (amend, rebase), a force push is required; `--force-with-lease` makes it safe by refusing if the remote moved since you last fetched.

NOTES

- Default `push.default simple` pushes the current branch to its upstream
- `-u` sets upstream so later `git push` / `pull` need no arguments
- `push.autoSetupRemote true` (Git 2.37+) does `-u` automatically
- Protected branches on GitHub/GitLab may block force pushes

PUSH VARIANTS

```
git push -u origin feature/login        # first push: create + track
git push                                # later pushes
git push origin HEAD                    # push current branch, same name
git push origin local:remote-name       # push to a different branch name
git push --dry-run                      # show what would happen

# ! [rejected] main -> main (non-fast-forward)
git pull --rebase && git push           # integrate, then push

git push --force-with-lease             # safe force after rebase/amend
git push --force-with-lease --force-if-includes   # extra safety
git push --force                        # overwrite unconditionally (avoid)

git push origin --delete feature/login  # delete a remote branch
git push --follow-tags                  # commits + annotated tags
git config --global push.autoSetupRemote true
```

CODE EXPLAINED

- → `-u` — remembers `origin/feature/login` as upstream
- → `HEAD` — the current branch, whatever its name
- → `local:remote` — refspec: source on the left, destination on the right
- → non-fast-forward — the remote has commits you don't
- → `--force-with-lease` — fails if the remote changed since your fetch
- → `--delete` — removes the branch on the server

FORCE PUSH OPTIONS

OPTION	SAFETY
`--force`	overwrites anything: risky
`--force-with-lease`	only if remote = your last fetch
`--force-if-includes`	also checks you saw the remote tip
`+branch refspec`	force for one ref only

PUSH ERRORS

MESSAGE	FIX
`non-fast-forward`	`git pull --rebase`, then push
`no upstream branch`	`git push -u origin HEAD`
`stale info (lease)`	`git fetch`, review, retry
`protected branch`	open a pull request instead
`Permission denied`	check SSH key / token (Sheet 23)

⚠ COMMON MISTAKES

- Using `--force` on a shared branch. It silently deletes teammates' commits
- Running `git fetch` right before `--force-with-lease`. The lease then matches whatever is there, which defeats it
- Pushing to `main` directly when the team uses PRs. Push a branch and open a PR

✓ TIPS & BEST PRACTICES

- Alias `pushf` to `push --force-with-lease`
- Set `push.autoSetupRemote true` and never type `-u` again
- Check `git log @{u}..` before pushing

QUICK SUMMARY

- `push -u origin <br>` — first push
- `--force-with-lease` — safe force
- `--delete` — remove remote branch
- rejected → pull --rebase → push

SSH, Credentials & Signing

ssh-keygen -t ed25519 · ssh -T git@github.com · ~/.ssh/config · HTTPS + tokens · credential.helper · Git Credential Manager · gpg.format ssh / commit -S

PROVING WHO YOU ARE

Hosts no longer accept account passwords for Git. Use either **SSH keys** (add your public key to your account) or **HTTPS with a token**, stored by a credential helper so you aren't prompted every time. Separately, **commit signing** proves a commit really came from you; Git 2.34+ can sign with the same SSH key.

NOTES

- Never share or commit the private key (`id_ed25519`)
- Personal access tokens replace passwords for HTTPS
- Git Credential Manager handles HTTPS + 2FA on all platforms
- Signed commits show as "Verified" on GitHub/GitLab

SSH KEYS, CREDENTIALS, SIGNING

```
ssh-keygen -t ed25519 -C "ada@example.com"   # creates ~/.ssh/id_ed25519(.pub)
ssh-add --apple-use-keychain ~/.ssh/id_ed25519   # macOS keychain
cat ~/.ssh/id_ed25519.pub                    # paste into GitHub > SSH keys
ssh -T git@github.com                        # "Hi ada! You've successfully…"

# ~/.ssh/config: pick a key per host
Host github.com
  IdentityFile ~/.ssh/id_ed25519
  AddKeysToAgent yes

git config --global credential.helper osxkeychain   # macOS (Windows: manager)

git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true      # sign every commit
git commit -S -m "Signed commit"             # sign one commit
git log --show-signature -1                  # verify
```

CODE EXPLAINED

- → `-t ed25519` — modern, short, fast key type
- → `ssh -T` — tests auth without opening a shell
- → `IdentityFile` — choose a key per host or alias
- → `credential.helper` — stores HTTPS tokens securely
- → `gpg.format ssh` — sign with an SSH key instead of GPG
- → `--show-signature` — checks signatures in the log

SSH VS HTTPS

ASPECT	SSH / HTTPS
Auth	key pair / token
URL	`git@host:u/r.git` / `https://…`
Firewalls	port 22 (or 443 alt) / 443
Setup	once per machine / helper stores token

TROUBLESHOOTING

SYMPTOM	CHECK
`Permission denied (publickey)`	key added to account? agent loaded?
`Asked for password`	use a token, not your password
`Wrong account used`	`~/.ssh/config` Host alias
`Debug SSH`	`ssh -vT git@github.com`
`Debug Git+SSH`	`GIT_SSH_COMMAND="ssh -v" git fetch`

⚠ COMMON MISTAKES

- Pasting the private key into GitHub. Only the `.pub` file goes there
- Using an HTTPS remote after setting up SSH. Switch with `git remote set-url origin git@…`
- Enabling `commit.gpgsign` without uploading the key as a *signing* key. Commits show "Unverified"

✓ TIPS & BEST PRACTICES

- Use a passphrase on keys and let the agent remember it
- Use `gh auth login` to set up GitHub HTTPS credentials quickly
- Two accounts on one host: add `Host github-work` aliases in `~/.ssh/config`

QUICK SUMMARY

- `ssh-keygen -t ed25519` — new key
- `ssh -T` — test the connection
- `credential.helper` — store tokens
- `gpg.format ssh` + `-S` — sign commits

Collaboration Workflows

feature branches · GitHub flow · Git flow · trunk-based · fork + sync upstream · gh pr create · gh pr checkout · release branches

HOW TEAMS USE GIT

A workflow is the team agreement about which branches exist and how changes reach `main`. Most teams use **short-lived feature branches + pull requests** (GitHub flow) or **trunk-based development** with tiny, frequent merges. Open-source projects add **forks**. Git flow's long-lived `develop` and release branches suit scheduled, versioned releases.

NOTES

- Keep branches small: hours to a few days
- Protect `main`: require PR review and passing CI
- Pick one merge style per repo: merge, squash, or rebase
- The `gh` CLI drives GitHub PRs from the terminal

FEATURE BRANCH + PR (GITHUB FLOW)

```
git switch main && git pull --ff-only
git switch -c feature/search          # 1. branch from fresh main
git add -p && git commit -m "Add search box"   # 2. small commits
git fetch && git rebase origin/main   # 3. stay current
git push -u origin HEAD               # 4. publish
gh pr create --fill                   # 5. open a pull request
gh pr checkout 87                     # review someone else's PR locally
gh pr merge --squash --delete-branch  # 6. merge when approved

# Fork workflow (open source)
gh repo fork org/app --clone          # fork + clone + add upstream remote
git fetch upstream
git rebase upstream/main              # sync your branch with the original
git push --force-with-lease
```

CODE EXPLAINED

- → `pull --ff-only` — refresh main without stray merges
- → `rebase origin/main` — resolve conflicts before review, not after
- → `gh pr create --fill` — title/body from your commits
- → `gh pr checkout` — fetches a PR into a local branch
- → `gh repo fork --clone` — sets `origin` (fork) and `upstream`
- → `--delete-branch` — cleans up after merging

BRANCHING MODELS

MODEL	SHAPE / BEST FOR
`GitHub flow`	main + short feature branches; continuous deploy
`Trunk-based`	tiny merges to main daily; feature flags
`Git flow`	main, develop, feature/, release/, hotfix/; versioned releases
`Forking`	PRs from personal forks; open source
`Release branches`	`release/2.1` for patches to old versions

PR MERGE STYLES

STYLE	RESULT ON MAIN
`Merge commit`	all commits + a merge commit
`Squash`	one commit per PR
`Rebase`	all commits, linear, no merge commit

⚠ COMMON MISTAKES

- Long-running feature branches. Conflicts grow every day; merge small pieces behind flags
- Working directly on `main` in a fork. Use a branch, so syncing upstream stays a fast-forward
- Mixing merge styles in one repo. History becomes hard to read and bisect

✓ TIPS & BEST PRACTICES

- Name branches by type: `feature/`, `fix/`, `chore/`
- Write PR descriptions that explain why and how to test
- Delete merged branches locally with `git fetch --prune` + `git branch -d`

QUICK SUMMARY

- branch → commit → push → PR → merge
- `gh pr create` / `checkout` / `merge`
- forks: `origin` + `upstream`
- short branches = small conflicts

Reset

MOVING THE BRANCH POINTER

`git reset <commit>` moves the **current branch** to another commit. The mode decides what else changes: `--soft` leaves the staging area and files alone, `--mixed` (the default) also resets the staging area, and `--hard` also overwrites your working files. Reset rewrites history, so use it on **local, unpushed** commits.

NOTES

- Commits you reset away stay in the reflog for ~90 days
- `--hard` discards uncommitted changes permanently
- `ORIG_HEAD` = where HEAD was before reset, merge, or rebase
- With a file path, reset only unstages: no pointer moves

RESET MODES IN PRACTICE

```
git reset --soft HEAD~1         # undo last commit, keep changes staged
git reset HEAD~1                # undo last commit, keep changes unstaged (--mixed)
git reset --hard HEAD~1         # undo last commit AND discard its changes
git reset --hard                # throw away all uncommitted changes
git reset --hard origin/main    # make the branch match the remote exactly
git reset --keep HEAD~1         # like --hard but stops if it would lose edits

git reset app.js                # unstage a file (= git restore --staged app.js)
git reset --hard ORIG_HEAD      # undo the last merge/rebase/reset

# squash the last 3 commits into one
git reset --soft HEAD~3
git commit -m "Add search feature"
```

CODE EXPLAINED

- → `--soft` — branch moves; index + files untouched
- → `--mixed` — branch + index reset; files untouched
- → `--hard` — branch, index, and files all reset
- → `--keep` — safer hard reset; aborts on conflicting local edits
- → `reset <file>` — copies HEAD's version into the index only
- → `--soft HEAD~3` + commit — quick squash without interactive rebase

WHAT EACH MODE CHANGES

MODE	BRANCH · INDEX · FILES
`--soft`	moves · kept · kept
`--mixed (default)`	moves · reset · kept
`--hard`	moves · reset · reset
`--keep`	moves · reset · kept if safe

RECIPES

GOAL	COMMAND
Redo last commit	`reset --soft HEAD~1`
Unstage everything	`git reset`
Match the remote	`reset --hard origin/main`
Undo a bad reset	`reset --hard HEAD@{1}`
Undo a merge	`reset --hard ORIG_HEAD`

⚠ COMMON MISTAKES

- `reset --hard` with uncommitted work. Uncommitted changes aren't in the reflog; they're gone
- Resetting commits that are already pushed. Your next push needs force and breaks teammates; use `revert`
- Typing `HEAD-1` . The syntax is `HEAD~1`

✓ TIPS & BEST PRACTICES

- Stash first if you're unsure: `git stash -u` then reset
- Prefer `--keep` over `--hard` when moving branches with local edits
- Check `git status` and `git log -1` right after a reset

QUICK SUMMARY

- `--soft` — keep staged
- `--mixed` — keep in files
- `--hard` — discard everything
- `ORIG_HEAD` — undo the last big move

Revert vs Reset vs Restore

CHOOSING THE RIGHT UNDO

Git has several "undo" commands that work at different levels. `git revert` adds a **new commit** that inverts an old one, so it's safe on shared branches. `git reset` **moves the branch** backward and is for local history. `git restore` only changes **files**. `commit --amend` replaces the most recent commit. Ask: is it pushed, and is it a commit or just files?

NOTES

- Revert never rewrites history
- Reverting a merge needs `-m 1` to name the mainline parent
- Reverting a revert re-applies the original change
- Restore and reset on files never create commits

REVERT EXAMPLES

```
git revert a1b2c3d                  # new commit that undoes a1b2c3d
git revert HEAD                     # undo the last commit, publicly
git revert --no-edit HEAD~2         # use the default message
git revert -n a1b2c3d b4e5f6a       # undo several, commit once yourself
git commit -m "Revert search feature"
git revert a1b2c3d..f6a7b8c         # revert a range (newest first)

git revert -m 1 9f8e7d6             # undo a merge commit, keep main's side
git revert --continue               # after resolving a conflict
git revert --abort

git commit --amend                  # fix the most recent commit instead
```

CODE EXPLAINED

- → `revert <sha>` — computes the inverse diff and commits it
- → `--no-edit` — skips the message editor
- → `-n` — stage the inverse changes without committing
- → `A..B` — reverts every commit after A up to B
- → `-m 1` — parent 1 (the branch merged into) is kept
- → `--amend` — only for the latest, unpushed commit

WHICH UNDO?

SITUATION	USE
Typo in last commit (not pushed)	`commit --amend`
Bad commit, already pushed	`git revert <sha>`
Bad commits, not pushed	`reset --soft` / `--hard`
Discard edits to a file	`git restore <file>`
Unstage a file	`git restore --staged <file>`
Old version of one file	`restore --source=<sha> <file>`
Bad merge, pushed	`git revert -m 1 <merge>`

WHAT EACH COMMAND TOUCHES

COMMAND	CHANGES
`revert`	adds a commit (history grows)
`reset`	branch pointer (+ index/files)
`restore`	files or index only
`commit --amend`	replaces HEAD commit
`rebase -i`	rewrites a series

⚠ COMMON MISTAKES

- Using `reset` to undo a pushed commit. Teammates still have it, and your force push confuses everyone
- Reverting a merge without `-m`. Git errors because it can't tell which side to keep
- Merging the branch again after reverting its merge. Git thinks it's already merged; revert the revert first

✓ TIPS & BEST PRACTICES

- Rule of thumb: pushed → `revert`; local → `reset` or `amend`
- Reference the reverted commit and the reason in the revert message
- Use `git revert -n` across several commits to ship one clean undo

QUICK SUMMARY

- `revert` — safe public undo
- `reset` — rewrite local history
- `restore` — file-level undo
- `--amend` — fix the last commit

Stash

git stash · -u · -m "msg" · stash list · pop vs apply · stash@{2} · stash show -p · stash branch

SHELVING WORK IN PROGRESS

`git stash` saves your uncommitted changes onto a stack and cleans the working tree, so you can switch branches, pull, or fix something urgent. Bring them back later with `pop` (apply and remove) or `apply` (apply and keep). By default stash only saves **tracked** files; add –u for new files.

NOTES

- Stashes are local; they are never pushed
- `stash@{0}` is always the most recent
- `pop` that conflicts keeps the stash so nothing is lost
- `--staged` (Git 2.35+) stashes only staged changes

STASH COMMANDS

```
git stash                          # stash tracked changes (= stash push)
git stash -u                       # include untracked files
git stash -a                       # include untracked + ignored
git stash push -m "wip: navbar"    # with a description
git stash push -p                  # choose hunks
git stash push -- src/nav.js       # only certain paths
git stash --keep-index             # stash unstaged, keep staged in place

git stash list                     # stash@{0}: On main: wip: navbar
git stash show -p stash@{1}        # view a stash's diff
git stash pop                      # apply newest + drop it
git stash apply stash@{2}          # apply, keep it in the list
git stash drop stash@{1}           # delete one
git stash clear                    # delete all (careful)
git stash branch fix-nav stash@{0} # new branch from the stash's base
```

CODE EXPLAINED

- → `-u` — untracked files too (new files are skipped otherwise)
- → `-m` — label it so `stash list` is readable
- → `-p` / `-- path` — partial stashes
- → `pop` vs `apply` — remove vs keep after applying
- → `show -p` — inspect before applying
- → `stash branch` — safest way to apply an old, conflicting stash

STASH CHEAT LIST

COMMAND	EFFECT
`stash`	save tracked changes
`stash -u`	+ untracked
`stash list`	show the stack
`stash pop`	apply + remove
`stash apply`	apply, keep
`stash drop`	remove one
`stash clear`	remove all

WHEN TO STASH

SITUATION	FLOW
`Urgent fix elsewhere`	`stash -u → switch → fix → back → pop`
`Pull with local edits`	`stash → pull → pop`
`Wrong branch`	`stash → switch → pop`
`Test staged only`	`stash --keep-index -u`

⚠ COMMON MISTAKES

- Forgetting `-u` . New files stay behind and follow you to the next branch
- Using stash as long-term storage. Stashes are easy to forget and lose; commit on a branch instead
- `git stash clear` by accident. Recovery means hunting dangling commits with `git fsck`

✓ TIPS & BEST PRACTICES

- Always add `-m` so you know what each stash is
- Set `git config --global rebase.autoStash true` to stash around rebases automatically
- Prefer `apply` + `drop` when a conflict is likely

QUICK SUMMARY

- `stash -u -m` — save everything, labeled
- `stash list` / `show -p` — inspect
- `pop` / `apply` — restore
- `stash branch` — apply safely

Reflog & Recovery

GIT'S SAFETY NET

The **reflog** records every position HEAD and each branch has had in *your* clone: commits, resets, rebases, checkouts. If a commit was ever committed, it is almost always recoverable: find it in the reflog and put a branch on it. Only uncommitted changes and very old unreachable objects are truly lost.

NOTES

- The reflog is local only; it isn't pushed or cloned
- Reachable entries expire after 90 days, unreachable after 30
- Deleting a branch prints its last hash: `(was a1b2c3d)`
- Dropped stashes can be found with `git fsck`

FINDING AND RESTORING LOST COMMITS

```
git reflog                        # HEAD history, newest first
# a1b2c3d HEAD@{0}: reset: moving to HEAD~3
# f6a7b8c HEAD@{1}: commit: Add search tests
git reflog show feature           # one branch's history
git log -g --oneline              # reflog in log format

git reset --hard HEAD@{1}         # undo the reset above
git branch rescue f6a7b8c         # put a branch on a lost commit
git switch -c rescue HEAD@{5}     # or create + switch

git branch -D feature             # Deleted branch feature (was 9e8d7c6).
git branch feature 9e8d7c6        # brought back

git fsck --lost-found             # dangling commits/blobs (e.g. dropped stash)
git show <dangling-sha>           # inspect before restoring
git stash apply <dangling-sha>    # re-apply a dropped stash
```

CODE EXPLAINED

- → `HEAD@{n}` — where HEAD was n moves ago
- → `reflog show <branch>` — that branch's own movements
- → `log -g` — walks the reflog with normal log formatting
- → `branch <name> <sha>` — makes a lost commit reachable again
- → `fsck --lost-found` — finds objects no ref or reflog points to
- → `stash apply <sha>` — works on a dropped stash commit

RECOVERY SCENARIOS

LOST BY	RECOVER WITH
`reset --hard`	`reflog` → `reset --hard HEAD@{n}`
`bad rebase`	`reset --hard ORIG_HEAD` or reflog
`deleted branch`	`branch <name> <sha>`
`amended commit`	`reflog` → old hash
`dropped stash`	`fsck --lost-found`
`uncommitted + reset`	not recoverable

EXPIRY SETTINGS

CONFIG	DEFAULT
`gc.reflogExpire`	90 days
`gc.reflogExpireUnreachable`	30 days
`gc.pruneExpire`	2 weeks
`git gc`	runs cleanup

⚠ COMMON MISTAKES

- Running `git gc --prune=now` while trying to recover. It can delete the objects you need
- Expecting the reflog on a new clone or a teammate's machine. It's per-clone history
- Panicking after a bad reset and running more commands. Stop, read `git reflog`, then act

✓ TIPS & BEST PRACTICES

- Before risky operations, drop a bookmark: `git branch backup`
- Use `git reflog --date=relative` to find "before lunch" states
- `ORIG_HEAD` is the fastest undo for the last reset/merge/rebase

QUICK SUMMARY

- `git reflog` — every HEAD move
- `HEAD@{n}` — reference an old state
- `branch <name> <sha>` — rescue
- `fsck --lost-found` — last resort

Common Errors & Fixes

detached HEAD · non-fast-forward rejected · unrelated histories · local changes would be overwritten · CRLF warnings · GIT_TRACE=1 · git gc / fsck

READING GIT'S MESSAGES

Most Git errors mean one of three things: your local state and the remote **diverged**, an operation would **overwrite uncommitted work**, or **authentication** failed. Git's message usually names the fix. When it doesn't, the trace variables below show exactly what Git is doing.

NOTES

- `git status` explains the current state and suggests commands
- Mid-operation? Look for `--continue` / `--abort` hints
- "warning" lines are informational; "fatal" stopped the command
- Search the exact message text; it's usually well documented

DIAGNOSTICS

```
git status                          # what state am I in?
GIT_TRACE=1 git pull                # trace commands Git runs internally
GIT_CURL_VERBOSE=1 git fetch        # HTTPS request/response details
GIT_SSH_COMMAND="ssh -v" git fetch # SSH handshake details
git config --list --show-origin    # which config is in effect
git remote -v                       # are the URLs what you expect?
git fsck                            # check repository integrity
git gc                              # pack and clean up objects
```

CODE EXPLAINED

- → `GIT_TRACE=1` — prints each internal command and timing
- → `GIT_CURL_VERBOSE` — debugs HTTPS auth and proxies
- → `GIT_SSH_COMMAND` — runs SSH with verbose output
- → `--show-origin` — finds a surprising setting
- → `fsck` / `gc` — integrity check and housekeeping

ERROR → FIX

MESSAGE (SHORT)	CAUSE	FIX
`You are in 'detached HEAD' state`	checked out a commit/tag	`git switch -c <branch>` to keep work
`rejected … (non-fast-forward)`	remote has commits you don't	`git pull --rebase`, then push
`has no upstream branch`	first push of a branch	`git push -u origin HEAD`
`refusing to merge unrelated histories`	two repos with no common commit	`--allow-unrelated-histories` (if intended)
`local changes … would be overwritten`	dirty files conflict with switch/pull	commit or `git stash -u`, then retry
`Permission denied (publickey)`	SSH key not set up or not loaded	`ssh -T git@github.com`; add key
`LF will be replaced by CRLF`	line-ending conversion	`.gitattributes` with `eol=lf`
`not a git repository`	outside a repo folder	`cd` into it or `git init`
`pathspec … did not match`	typo, or file not tracked	check `git status`; quote globs

⚠ COMMON MISTAKES

- Fixing a rejected push with `--force`. You overwrite others' work; integrate first
- Using `--allow-unrelated-histories` to silence an error. It's for intentionally joining two repos
- Ignoring "detached HEAD". Commits made there are easy to lose

✓ TIPS & BEST PRACTICES

- Read the full message: Git usually prints the exact command to run
- Run `git status` after every error before trying anything else
- Keep Git updated: newer versions give clearer errors and hints

QUICK SUMMARY

- `git status` — first diagnostic
- `GIT_TRACE=1` — see what Git does
- diverged → `pull --rebase`
- dirty tree → commit or stash

Bisect & Worktrees

git bisect start · good / bad / skip · bisect run <script> · bisect reset · git worktree add ../dir <branch> · worktree list / remove · worktree prune · one branch per worktree

HUNTING BUGS AND WORKING IN PARALLEL

`git bisect` binary-searches history to find the **first bad commit**: you mark one good and one bad commit, and Git checks out midpoints until it isolates the culprit, about $\log_2(n)$ steps. **Worktrees** give one repository several working folders, each on its own branch, so you can review a PR or hotfix without stashing your current work.

NOTES

- 1,000 commits take about 10 bisect steps
- `bisect run` exit codes: 0 good, 125 skip, 1–127 bad
- Worktrees share one `.git` object store, so they're cheap
- A branch can be checked out in only one worktree at a time

BISECT AND WORKTREE COMMANDS

```
git bisect start
git bisect bad                    # current commit is broken
git bisect good v1.4.0            # this tag worked
git bisect good   # / bad / skip  # repeat at each step Git checks out
git bisect run npm test           # or automate with any script
# a1b2c3d is the first bad commit
git bisect log > bisect.txt       # save the session
git bisect reset                  # return to where you started

git worktree add ../app-hotfix hotfix/2.1     # existing branch
git worktree add -b review-87 ../app-review origin/main  # new branch
git worktree list                 # all worktrees + their branches
git worktree remove ../app-review # delete the folder + registration
git worktree prune                # forget folders you deleted manually
```

CODE EXPLAINED

- → `bisect bad` / `good` — set the search bounds
- → `bisect skip` — commit can't be tested (doesn't build)
- → `bisect run <cmd>` — Git marks each step from the exit code
- → `bisect reset` — always finish with this
- → `worktree add <path> <branch>` — new folder on that branch
- → `-b <name>` — create the branch at the same time

BISECT EXTRAS

COMMAND	EFFECT
`bisect start BAD GOOD`	set both bounds at once
`bisect view`	show remaining suspects
`bisect replay file`	rerun a saved log
`--first-parent`	only test mainline commits
`--term-old / --term-new`	rename good/bad (e.g. fast/slow)

WORKTREE EXTRAS

COMMAND	EFFECT
`worktree add --detach`	no branch, just a commit
`worktree lock <path>`	protect from prune
`worktree move <a> <b>`	relocate a worktree
`worktree repair`	fix links after moving

⚠ COMMON MISTAKES

- Forgetting `git bisect reset`. You're left on a detached midpoint commit
- A `bisect run` script that fails for unrelated reasons. Make it exit 125 when it can't build
- Deleting a worktree folder with `rm` only. Run `git worktree prune` or Git thinks the branch is still checked out

✓ TIPS & BEST PRACTICES

- Write a tiny repro script before bisecting; automated runs are fast and reliable
- Keep worktrees as siblings: `../app-review`, `../app-hotfix`
- Use a worktree to run long tests on one branch while coding on another

QUICK SUMMARY

- `bisect start` / `good` / `bad` — search
- `bisect run` — automate
- `worktree add` — parallel checkout
- `worktree remove` / `prune` — clean up

Submodules & Subtrees

submodule add · update --init --recursive · .gitmodules · submodule foreach · git subtree add --prefix · subtree pull --squash

REPOSITORIES INSIDE REPOSITORIES

A **submodule** embeds another repo at a pinned commit: the parent stores only the URL (in `.gitmodules`) and a commit hash. That keeps histories separate but requires extra commands to clone and update. A **subtree** copies the other project's files into a folder of your repo, so clones just work, at the cost of a larger history.

NOTES

- A submodule is pinned to a commit, not a branch
- Fresh clones have empty submodule folders until you init them
- `submodule.recurse true` makes pull/switch update submodules
- `git subtree` ships with most Git installs (contrib)

SUBMODULE AND SUBTREE COMMANDS

```
git submodule add https://github.com/org/lib.git vendor/lib
git commit -m "Add lib submodule"           # commits .gitmodules + pointer

git clone --recurse-submodules <url>        # clone with submodules
git submodule update --init --recursive     # after a plain clone
git submodule update --remote --merge       # move to the latest upstream commit
git submodule status                        # pinned hash per submodule
git submodule foreach 'git pull'            # run a command in each
git config --global submodule.recurse true

git rm vendor/lib                           # remove a submodule (then commit)

git subtree add --prefix=vendor/lib https://github.com/org/lib.git main --squash
git subtree pull --prefix=vendor/lib https://github.com/org/lib.git main --squash
git subtree push --prefix=vendor/lib <url> my-fixes   # send changes upstream
```

CODE EXPLAINED

- → `submodule add` — clones it and records URL + path
- → `update --init --recursive` — fills in nested submodules too
- → `--remote` — fetch the tracked branch instead of the pinned hash
- → `foreach` — runs a shell command in every submodule
- → `subtree add --prefix` — imports files into that folder
- → `--squash` — imports history as one commit

SUBMODULE VS SUBTREE

ASPECT	SUBMODULE / SUBTREE
Stored as	pointer + URL / real files
Clone	needs extra step / just works
Update	`update --remote` / `subtree pull`
History	separate / merged in
Best for	large, independently versioned deps / small vendored code

.GITMODULES ENTRY

KEY	MEANING
`[submodule "lib"]`	section name
`path`	folder in your repo
`url`	where to clone from
`branch`	branch for `--remote`

⚠ COMMON MISTAKES

- Committing inside a submodule without committing the new pointer in the parent. Others still get the old commit
- Working in a submodule on detached HEAD. Switch to a branch before editing
- Mixing `--squash` and non-squash subtree pulls. Pick one and stick with it

✓ TIPS & BEST PRACTICES

- Add `--recurse-submodules` to CI clone steps
- Show submodule changes in status: `git config status.submoduleSummary true`
- Consider your package manager before either; it's often simpler

QUICK SUMMARY

- `submodule add` — pin another repo
- `update --init --recursive` — populate
- `subtree add --prefix` — vendor files
- `subtree pull` — update vendored copy

Large Repos & Big Files

clone --filter=blob:none · sparse-checkout set · fetch --unshallow · git lfs install / track · git maintenance start · scalar

SCALING GIT

Git copies all history by default, which gets slow for huge monorepos or repos full of binaries. **Partial clones** skip file contents until needed, **sparse checkout** materializes only the folders you work in, and **Git LFS** stores large binaries outside the repo, leaving small pointer files behind. Background **maintenance** keeps big repos fast.

NOTES

- Partial clone keeps full history; shallow clone cuts it
- Git LFS is a separate install: `brew install git-lfs`
- GitHub warns at 50 MB and blocks files over 100 MB
- `scalar` (bundled since 2.38) sets all of this up at once

BIG-REPO TOOLS

```
git clone --filter=blob:none <url>        # blobs downloaded on demand
git clone --filter=tree:0 <url>           # even lighter (CI builds)
git clone --depth 1 <url>                 # shallow: latest commit only
git fetch --unshallow                     # convert shallow to full

git clone --sparse <url> && cd repo       # start with root files only
git sparse-checkout set apps/web libs/ui  # materialize just these folders
git sparse-checkout add docs              # add another folder
git sparse-checkout disable               # back to everything

git lfs install                           # once per machine
git lfs track "*.psd" "*.mp4"             # writes rules to .gitattributes
git add .gitattributes design.psd && git commit -m "Track PSDs in LFS"
git lfs ls-files                          # files stored in LFS
git lfs migrate import --include="*.zip"  # move existing files (rewrites history)

git maintenance start                     # scheduled background optimization
```

CODE EXPLAINED

- → `blob:none` — commits + trees now, file contents lazily
- → `--sparse` + `sparse-checkout set` — only listed folders on disk
- → `lfs track` — adds `filter=lfs` lines to `.gitattributes`
- → `lfs ls-files` — confirms what LFS manages
- → `lfs migrate import` — moves history into LFS (force push needed)
- → `maintenance start` — registers prefetch, gc, commit-graph tasks

WHICH OPTION?

PROBLEM	TOOL
Clone takes forever	`--filter=blob:none`
CI only needs latest	`--depth 1`
Huge monorepo, one team	`sparse-checkout`
Large binaries	Git LFS
Slow status/log	`git maintenance start`
All of the above	`scalar clone <url>`

MEASURING SIZE

COMMAND	SHOWS
`git count-objects -vH`	repo size on disk
`git lfs ls-files -s`	LFS files + sizes
`git rev-list --objects --all`	every object (pipe to find big ones)
`du -sh .git`	total .git size

⚠ COMMON MISTAKES

- Committing a large binary, then deleting it. It stays in history forever; rewrite with filter-repo (Sheet 34)
- Running `git lfs track` without committing `.gitattributes`. Teammates add the files as normal blobs
- Using `--depth 1` for development work. Blame, log, and merges lose history

✓ TIPS & BEST PRACTICES

- Set up LFS before the first binary is committed
- Use partial clone over shallow clone for day-to-day work
- Add `.gitattributes` LFS rules in the very first commit of asset-heavy repos

QUICK SUMMARY

- `--filter=blob:none` — partial clone
- `sparse-checkout set` — some folders
- `git lfs track` — big binaries
- `maintenance start` — keep it fast

Hooks

AUTOMATING CHECKS

Hooks are scripts Git runs at specific points, such as before a commit or before a push. A hook that exits **non-zero** stops the operation, so they're used for linting, formatting, test runs, and message checks. Hooks in `.git/hooks/` aren't committed, so teams share them via `core.hooksPath` or a hook manager.

NOTES

- Hooks must be executable: `chmod +x`
- Any language works; the shebang line picks the interpreter
- Client hooks can be skipped; enforce rules in CI or server hooks too
- `.sample` files in `.git/hooks/` are examples

A PRE-COMMIT HOOK

```
#!/bin/sh
# .githooks/pre-commit — block commits with lint errors or debug code
npm run lint --silent || exit 1
if git diff --cached | grep -q "console.log"; then
  echo "Remove console.log before committing" >&2
  exit 1
fi

chmod +x .githooks/pre-commit
git config core.hooksPath .githooks   # use the committed hooks folder
git commit --no-verify -m "WIP"       # skip pre-commit + commit-msg once

# Hook managers
pip install pre-commit && pre-commit install    # .pre-commit-config.yaml
npx husky init                                  # .husky/ for JS projects
```

CODE EXPLAINED

- → `exit 1` — non-zero exit aborts the commit
- → `git diff --cached` — inspect only what's being committed
- → `>&2` — print the reason to stderr
- → `core.hooksPath` — point Git at a versioned hooks folder
- → `--no-verify` / `-n` — bypass hooks for one commit
- → pre-commit / Husky — install and share hooks automatically

CLIENT-SIDE HOOKS

HOOK	RUNS
`pre-commit`	before the message prompt; lint/format
`prepare-commit-msg`	to pre-fill the message
`commit-msg`	validate message (gets file path $1)
`post-commit`	after commit; notifications
`pre-rebase`	before a rebase starts
`post-checkout`	after switch/checkout
`pre-push`	before push; run tests

SERVER-SIDE HOOKS

HOOK	RUNS
`pre-receive`	before accepting a push
`update`	once per pushed branch
`post-receive`	after push; deploy/notify

⚠ COMMON MISTAKES

- Forgetting `chmod +x` . Git silently ignores non-executable hooks
- Slow pre-commit hooks running the full test suite. People start using `--no-verify` ; keep them fast
- Relying on client hooks for policy. Anyone can skip them; enforce in CI

✓ TIPS & BEST PRACTICES

- Check only staged files to keep hooks fast (lint-staged, pre-commit)
- Put heavier checks in `pre-push` instead of `pre-commit`
- Commit hooks in `.githooks/` and document the `core.hooksPath` step

QUICK SUMMARY

- `pre-commit` — lint before commit
- `commit-msg` — enforce message format
- `core.hooksPath` — share hooks
- `--no-verify` — skip once

Rewriting History & Removing Secrets

git filter-repo · --path / --invert-paths · --replace-text · BFG · force-push afterward · rotate leaked secrets · filter-branch (deprecated)

PURGING FILES FROM EVERY COMMIT

When a secret or huge file was committed, deleting it in a new commit isn't enough: it's still in history. `git filter-repo` rewrites **every commit** to remove paths or replace text, producing new hashes for the whole history. Afterward everyone must re-clone. Most importantly, a leaked secret must be **rotated** first, because it may already have been copied.

NOTES

- filter-repo is a separate install: `pip install git-filter-repo`
- It refuses to run on a non-fresh clone without `--force`
- It removes the `origin` remote as a safety measure
- `git filter-branch` is deprecated: slow and error-prone

FILTER-REPO AND BFG

```
# 0. Rotate the leaked key/password FIRST
git clone --mirror git@github.com:org/app.git && cd app.git   # fresh copy

git filter-repo --path secrets.env --invert-paths   # delete a file everywhere
git filter-repo --path-glob "*.mp4" --invert-paths  # delete by pattern
git filter-repo --strip-blobs-bigger-than 50M       # drop huge blobs
git filter-repo --replace-text ../replacements.txt  # scrub strings
# replacements.txt:   sk_live_abc123==>***REMOVED***
git filter-repo --path src/ --path-rename src/:lib/ # keep only src/, rename it

git remote add origin git@github.com:org/app.git
git push --force --mirror origin                    # rewrite the server

# BFG alternative (Java)
bfg --delete-files id_rsa
bfg --replace-text passwords.txt
git reflog expire --expire=now --all && git gc --prune=now --aggressive
```

CODE EXPLAINED

- → `--mirror` — bare clone with every ref, ideal for rewriting
- → `--path X --invert-paths` — keep everything except X
- → `--strip-blobs-bigger-than` — remove large files by size
- → `--replace-text` — `old==>new` lines, `regex:` allowed
- → `push --force --mirror` — replaces all branches and tags
- → `reflog expire` + `gc --prune=now` — purge old objects locally (BFG)

LEAKED SECRET CHECKLIST

STEP	ACTION
1	revoke / rotate the secret
2	rewrite history (filter-repo)
3	force-push all branches + tags
4	ask the host to purge cached views/PR refs
5	everyone re-clones (don't merge old clones)
6	add the file to `.gitignore` + secret scanning

TOOLS

TOOL	NOTES
`git filter-repo`	recommended; fast, flexible
`BFG Repo-Cleaner`	simple deletes/replacements; needs Java
`git filter-branch`	deprecated; avoid
`rebase -i`	fine for a few recent local commits

⚠ COMMON MISTAKES

- Rewriting history but not rotating the secret. Forks, clones, and caches may still have it
- Letting teammates pull into old clones. Merging old history brings the secret right back
- Running filter-repo on your only copy. Work on a fresh mirror clone and keep a backup

✓ TIPS & BEST PRACTICES

- Turn on secret scanning / push protection on your host
- Use `.env.example` with placeholders; ignore real `.env` files
- Run `git log --all -- secrets.env` afterward to confirm it's gone

QUICK SUMMARY

- rotate the secret first
- `filter-repo --invert-paths` — remove files
- `--replace-text` — scrub strings
- force-push + everyone re-clones

Sheet 35

Quick Reference — Everyday Commands

setup & config · stage & commit · diff & log · revision syntax · branch & switch · tags · fetch / pull / push

SETUP & CONFIG

COMMAND	DOES	#
`git config --global user.name "N"`	set your name	02
`git config --global user.email "e"`	set your email	02
`git config --global init.defaultBranch main`	default branch name	02
`git config --list --show-origin`	all settings + source file	03
`git config --global alias.st "status -sb"`	create an alias	03
`git help <cmd>` · `git <cmd> -h`	manual · short help	03
`git init` · `git init -b main`	new repository	04
`git clone <url> [dir]`	copy a repository	04
`git clone --depth 1 <url>`	shallow clone	04
`git check-ignore -v <path>`	why is it ignored?	05

STAGE & COMMIT

COMMAND	DOES	#
`git status -sb`	short status + branch	06
`git add <file>` · `git add -A`	stage file · stage all	06
`git add -p`	stage hunks interactively	06
`git restore --staged <file>`	unstage	06
`git rm --cached <file>`	untrack, keep on disk	05
`git commit -m "msg"`	commit staged changes	07
`git commit -am "msg"`	stage tracked + commit	07
`git commit --amend --no-edit`	add to last commit	07
`git restore <file>`	discard unstaged edits	09
`git clean -n` · `git clean -fd`	preview · delete untracked	09

DIFF & INSPECT

COMMAND	DOES	#
`git diff`	unstaged changes	08
`git diff --staged`	staged changes	08
`git diff main...feature`	branch changes since fork	08
`git diff --stat`	per-file summary	08
`git show <sha>`	commit + diff	13
`git show <rev>:<path>`	file at a revision	13
`git blame -L 10,30 <file>`	who changed these lines	13
`git grep -n "text"`	search tracked files	13
`git describe --tags`	name from nearest tag	13

LOG & SEARCH HISTORY

COMMAND	DOES	#
`git log --oneline --graph --all`	visual history	10
`git log -p -- <file>`	file history with diffs	10
`git log --follow -- <file>`	across renames	10
`git log main..feature`	on feature, not main	10
`git log --author="Ada"`	by author	11
`git log --grep="fix"`	search messages	11
`git log -S "func"`	string added/removed	11
`git log --since="2 weeks ago"`	date filter	11
`git shortlog -sn`	commits per author	10

BRANCHES & TAGS

COMMAND	DOES	#
`git branch -vv`	branches + tracking	14
`git switch -c <name>`	create + switch	14
`git switch <name>` · `git switch -`	switch · previous	14
`git branch -m <old> <new>`	rename	14
`git branch -d <name>` · `-D`	delete · force	14
`git branch --merged main`	safe to delete	14
`git tag -a v1.0 -m "msg"`	annotated tag	15
`git push --follow-tags`	push commits + tags	15
`git tag -d v1.0`	delete local tag	15

REMOTES, FETCH, PULL, PUSH

COMMAND	DOES	#
`git remote -v`	list remotes	21
`git remote add upstream <url>`	add a remote	21
`git fetch --all --prune`	download + prune	21
`git pull --rebase`	fetch + rebase	21
`git pull --ff-only`	fast-forward only	21
`git push -u origin HEAD`	first push of a branch	22
`git push --force-with-lease`	safe force push	22
`git push origin --delete  `	delete remote branch	22
`git log @{u}..`	unpushed commits	12

REVISION SYNTAX

SYNTAX	MEANS	#
`HEAD~2` · `HEAD^2`	2 back · merge's 2nd parent	12
`A..B` · `A...B`	in B not A · either, not both	12
`@{u}` · `@{-1}`	upstream · previous branch	12
`HEAD@{2}` · `main@{yesterday}`	reflog positions	12

STATUS CODES (-S)

CODE	MEANS	#
`??` · `A_`	untracked · new, staged	06
`M_` · `_M` · `MM`	staged · unstaged · both	06
`D_` · `R_`	deleted · renamed (staged)	06
`UU`	merge conflict	17

⚠ COMMON MISTAKES

- `git diff` shows only unstaged changes; use `--staged`
- `git commit -a` skips new files
- Tags aren't pushed by plain `git push`

✓ TIPS & BEST PRACTICES

- `#` column = the sheet with full details
- Run `git status` before and after each command
- Alias the commands you type most

QUICK SUMMARY

- stage → commit → push
- fetch + rebase to stay current
- branch for every change

Sheet 36

Quick Reference — Merge, Undo & Advanced

merge & rebase · cherry-pick · stash · reset / revert / restore · reflog · bisect / worktree · Oops undo table

MERGE & CONFLICTS

COMMAND	DOES	#
`git merge <branch>`	merge into current	16
`git merge --no-ff · --ff-only`	force / forbid merge commit	16
`git merge --squash <branch>`	stage as one change	16
`git merge --abort`	cancel the merge	16
`git diff --name-only --diff-filter=U`	list conflicted files	17
`git checkout --ours|--theirs <f>`	take one side	17
`git add <f> · --continue`	mark resolved · finish	17
`git mergetool`	open merge tool	17

REBASE & CHERRY-PICK

COMMAND	DOES	#
`git rebase main`	replay branch onto main	18
`git rebase --continue|--skip|--abort`	conflict controls	18
`git rebase --onto <new> <old>  `	transplant commits	18
`git rebase -i HEAD~5`	edit last 5 commits	19
`git commit --fixup=<sha>`	mark a fix	19
`git rebase -i --autosquash main`	apply fixups	19
`git cherry-pick -x <sha>`	copy a commit	20
`git format-patch -3 · git am`	export · apply patches	20

UNDO

COMMAND	DOES	#
`git reset --soft HEAD~1`	uncommit, keep staged	25
`git reset HEAD~1`	uncommit, keep in files	25
`git reset --hard HEAD~1`	uncommit + discard	25
`git reset --hard ORIG_HEAD`	undo merge/rebase/reset	25
`git revert <sha>`	inverse commit (safe if pushed)	26
`git revert -m 1 <merge>`	undo a merge commit	26
`git restore --source=<sha> <f>`	old version of a file	09
`git reflog`	every HEAD move	28
`git branch rescue <sha>`	recover a lost commit	28

STASH & ADVANCED

COMMAND	DOES	#
`git stash -u -m "msg"`	shelve incl. untracked	27
`git stash list · pop · apply`	view · restore	27
`git bisect start · bad · good <rev>`	find a bad commit	30
`git bisect run <cmd> · reset`	automate · finish	30
`git worktree add ../dir  `	parallel checkout	30
`git submodule update --init --recursive`	populate submodules	31
`git sparse-checkout set <dirs>`	only some folders	32
`git lfs track "*.psd"`	big files via LFS	32
`git filter-repo --path f --invert-paths`	purge a file from history	34

OOPS! — NOT PUSHED YET

SITUATION	FIX
Typo in last commit message	`git commit --amend`
Forgot a file in last commit	`git add f && git commit --amend --no-edit`
Committed to the wrong branch	`git branch new && git reset --hard HEAD~1 && git switch new`
Want to redo the last commit	`git reset --soft HEAD~1`
Messy commits before a PR	`git rebase -i main`
Reset/rebase went wrong	`git reflog` → `git reset --hard HEAD@{n}`
Deleted a branch	`git branch <name> <sha-from-message>`

OOPS! — ALREADY PUSHED / OTHER

SITUATION	FIX
Bad commit on main	`git revert <sha>` + push
Bad merge on main	`git revert -m 1 <merge>`
Pushed a secret	rotate it, then `filter-repo` (34)
Push rejected	`git pull --rebase` → push
Need to update my PR after rebase	`git push --force-with-lease`
Committed in detached HEAD	`git switch -c <branch>`
Lost uncommitted work	only if stashed/staged: `git fsck --lost-found`

⚠ COMMON MISTAKES

- `reset` on pushed commits; use `revert`
- `--force` instead of `--force-with-lease`
- `--ours` / `--theirs` swap meaning in a rebase

✓ TIPS & BEST PRACTICES

- Make a `backup` branch before risky operations
- Enable `rerere` and `zdiff3` for easier conflicts
- When stuck, `git status` names the next command

QUICK SUMMARY

- pushed → revert · local → reset
- lost commits live in the reflog
- every operation has `--abort`